Harness Engineering for Self-Improvement

Date: July 4, 2026 | Estimated Reading Time: 31 min | Author: Lilian Weng
Original: https://lilianweng.github.io/posts/2026-07-04-harness/

▼ Table of Contents

- Harness Design Patterns
  - Pattern 1: Workflow Automation
  - Pattern 2: File System as Persistent Memory
  - Pattern 3: Sub-agent and Backend Jobs
  - Case study: Coding Agent Harness
  - Harness Layer vs Core Intelligence?
- Harness Optimization
  - Context Engineering
  - Workflow Design
  - Self-Improving Harness
  - Evolutionary Search
  - Joint Optimization with Model Weights
- Future Challenges
- Citation
- Appendix: Some useful benchmarks
- References

The concept of **recursive self-improvement (RSI)** dates back to I. J. Good (1965), where he defined an "ultraintelligent machine" as a system that can surpass humans in all intellectual activities and design better machines to improve itself. Yudkowsky (2008) used the phrase "recursive self-improvement" for a specific feedback loop: an AI uses its current intelligence to improve the cognitive machinery that produces its intelligence.

This feedback loop in modern AI may indicate the model rewriting its own weights directly, or more broadly the model improves the *training pipeline* and the *deployment system*, which in turn enables a better successor model with improved performance across economically valuable tasks. The speed of research development in AI has been shown to drastically accelerated in frontier labs (Anthropic; OpenAI).

I explicitly mention *"deployment system"* because the layer between the raw model and the real-world context seems to be as important as the model's raw intelligence (i.e. the evals right after pretraining). Harnesses are important components of AI deployment, as shown by successful coding agent products such as Claude Code and Codex. A **harness** is the system surrounding a base model that orchestrates execution and decides how the model thinks and plans, calls tools and acts, perceives and manages context, stores artifacts, and evaluates results.

This one post will focus on research around harness engineering and how it contributes to RSI. Much recent work on auto-research, self-improving agents, and evolutionary program search can be organized around this question. Other work on model self-play, synthetic data, test-time training and a broader theme of continual learning also matches the RSI vision (e.g. Yuan et al. 2024, Chen et al. 2024), Zhao et al. 2025, Choi et al. 2026)) but they will not be the focus of this post.

用于自我改进的运行框架工程

日期：7月4日， 2026 | 预计阅读时间：31分钟 | 作者：Lilian Weng原文：
https://lilianweng.github.io/posts/2026-07-04-harness/

▼ 目录

- 智能体运行框架设计模式模式1：工作流自动
  - 化模式2：以文件系统作为持久记忆模式3：
  - 子智能体与后台任务案例研究：编程智能体
  - 运行框架运行框架层与核心智能？运行框架
  - 优化上下文工程工作流设计自我改进的智能
  - 体运行框架演化搜索与模型权重的联合优化
- 未来挑战 引文 附录：一些实用的基准测试
  - 参考文献

这一概念——**递归自我改进（递归自我改进）**——最早可追溯至I. J. Good（1965年），他将“超智能机器”定义为一种在所有智力活动中都能超越人类，并能通过设计更好的机器来改进自身的系统。尤德考斯基（2008年）用“递归自我改进”一词描述一种特定的反馈回路：人工智能利用其当前的智能，改进产生其智能的认知机制。

现代人工智能中的这种反馈回路，可能意味着模型直接改写自身的权重；更广泛地说，也可能是模型改进了训练流水线和部署系统，进而催生出更好的后继模型，使其在具有经济价值的各类任务中表现更佳。已有证据表明，前沿人工智能实验室的人工智能研发速度已大幅加快（Anthropic；OpenAI）。

我特意提到“部署系统”，是因为原始模型与现实世界上下文之间的这一层，似乎与模型本身的智能水平（即预训练刚结束时的评测表现）同样重要。智能体运行框架是人工智能部署的重要组成部分，Claude Code 和 Codex 等成功的编程智能体产品就证明了这一点。**运行框架**是围绕基础模型构建的系统，负责统筹执行流程，并决定模型如何思考与规划、调用工具与采取行动、感知与管理上下文、存储产物以及评估结果。

本文将聚焦于运行框架工程相关研究，以及它如何助力递归自我改进。近期关于自动化研究、自我改进智能体和演化程序搜索的许多工作，都可以围绕这一问题加以梳理。其他关于模型自博弈、合成数据、测试时训练以及持续学习这一更广泛主题的工作，也契合递归自我改进的愿景（例如 Yuan 等人，2024年、Chen 等人，2024年），Zhao 等人，2025年、Choi 等人，2026年）），但这些并非本文的重点。

Harness Design Patterns

Compared with early agent frameworks, "agent = LLM + memory + tools + planning + action", harnesses engineering additionally include *workflow design (e.g. loop engineering), evaluation, permission controls, and persistent state management*. It is no longer only prompt templates, but closer to runtime and software system design: how the model observes, acts, memorizes, checks itself, and improves.

The design should be deliberately simple and generic to enable generalization, likely with reference to existing software engineering practices to benefit from prertaining knowlege. There is also a strong analogy between operating systems and harnesses. Similar to an OS, a harness should encapsulate complicated logic while keeping the interface simple. Meanwhile, configs, tool interfaces and other protocols may gradually become standardized across the industry.

Pattern 1: Workflow Automation

Defining a workflow in which the model can operate, test, and iterate is a key design for automation. Karpathy's autoresearch repo (https://github.com/karpathy/autoresearch) is a clean example of how such a workflow can be constructed. A common workflow follows a goal-oriented loop of plan, execute, observe/test, improve, and execute again *until* the goal is achieved. The process may trigger proactive requests to users for clarity in task specification or execution preference.

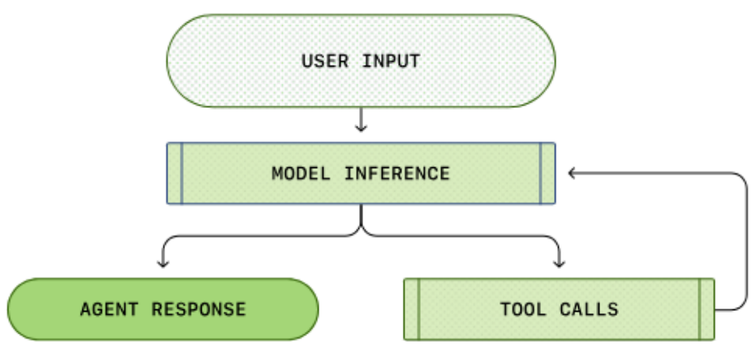


Figure 1: A simplified Codex agent loop: the agent calls tools and tool responses affect the model's next generation.
(Image source: OpenAI codex agent post)

The workflow graph also emphasizes the model analyzing its own trajectories and failure cases and then iterating on its progress through an "agent runtime" rather than a static prompt template.

Pattern 2: File System as Persistent Memory

A recurring pattern in long-horizon agent systems is simple control over rich states and artifacts. A harness should not carry the entire workflow and all logs in context; instead, it should keep durable state in files. In long-horizon agentic rollout, artifacts such as experiment logs, code diffs, paper summaries, error traces, and past rollout trajectories often grow much longer than the context window that the model has trained for.

智能体运行框架设计模式

相比于早期的智能体框架，即“智能体 = 大语言模型 + 记忆 + 工具 + 规划 + 行动”，运行框架工程还包括工作流设计（例如循环工程）、评估、权限控制和持久状态管理。它不再仅限于提示词模板，而是更接近运行时和软件系统设计：模型如何观察、行动、记忆、自检和改进。

设计应有意保持简单、通用，以实现泛化，并可参考现有的软件工程实践，从而利用预训练知识。此外，操作系统与智能体运行框架之间也有很强的相似性。与操作系统类似，运行框架应封装复杂逻辑，同时保持接口简单。与此同时，配置、工具接口和其他协议可能会逐渐在整个行业内实现标准化。

模式 1：工作流自动化

定义一个让模型能够在其中操作、测试和迭代的工作流，是实现自动化的关键设计。Karpathy 的 autoresearch 代码仓库（https://github.com/karpathy/autoresearch）清晰地展示了如何构建这样的工作流。常见的工作流遵循一个目标导向循环：规划、执行、观察/测试、改进，然后再次执行，直到目标达成。在此过程中，模型可能会主动请求用户澄清任务规格说明或执行偏好。

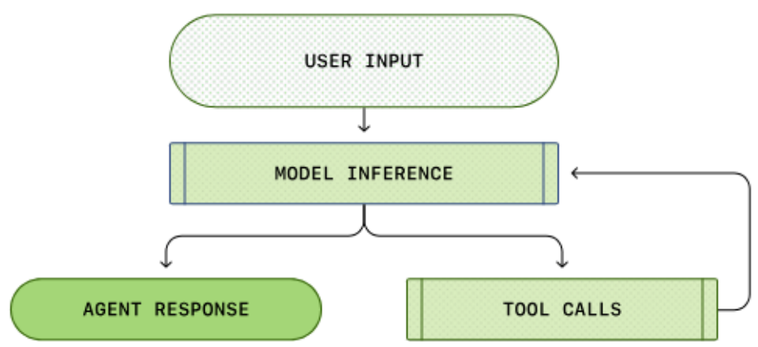


图 1：简化的 Codex 智能体循环：智能体调用工具，工具响应会影响模型的下一次生成。（图片来源：OpenAI 关于 Codex 智能体的文章）

该工作流图还强调，模型应分析自身的轨迹和失败案例，然后通过“智能体运行时”而非静态提示词模板来迭代改进。

模式 2：将文件系统用作持久记忆

长时程智能体系统中反复出现的一种模式，是以简单的控制机制管理丰富的状态和产物。运行框架不应将整个工作流和所有日志都放在上下文中，而应将持久状态保存在文件中。在长时程智能体轨迹展开过程中，实验日志、代码差异、论文摘要、错误追踪记录以及过去的展开轨迹等产物，其长度往往会增长到远超模型训练时所用的上下文窗口。

Learning how to read, write, and edit the file system (commonly via `bash` commands) is a foundation skill for LLMs, and thus managing persistent memory in the simple form of files naturally benefits from improvements in core model capability.

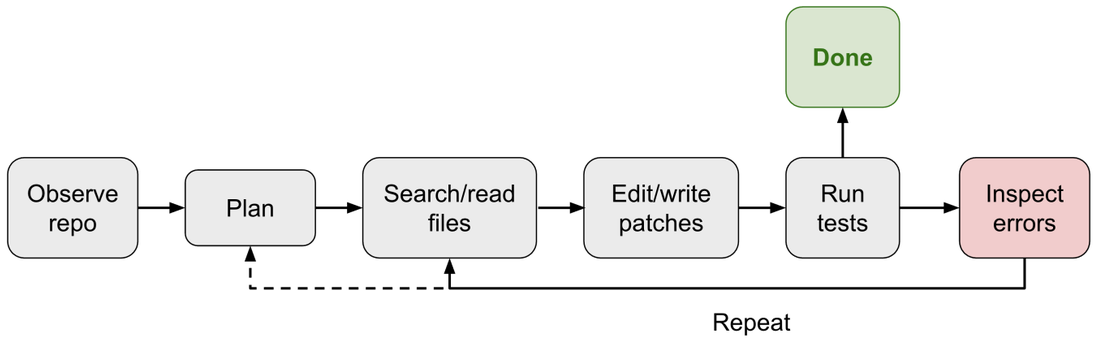
Pattern 3: Sub-agent and Backend Jobs

A harness can spawn multiple subagents to execute in parallel and monitor backend jobs. This is useful when the main agent needs to search multiple hypotheses, run experiments concurrently, or delegate isolated subtasks without polluting the main context. The parent agent then needs a small process manager: launch jobs, inspect logs, cancel failed runs, and merge results back into the main agent thread.

The key design choice is to make parallelism explicit and inspectable. If subagent outputs only live in a transient chat context, they quickly become obselete and hidden. If they are stored as files, logs, and status records, the model can recover after interruptions and reason over its own execution history.

Case study: Coding Agent Harness

The core interface of mainstream coding agents has become stabilized across Claude Code, Codex, OpenCode, and Cursor-style agents. They commonly use a loop like:



With access to a set of tools, the coding agent is able to develop and debug issues in a given repository, similar to how human developers are equipped with IDEs.

(Not a comprenhensive list; shown for demonstration. Read this if interested.)

Group	Tool definitions
File system	- File discovery: `glob` , `grep` , `ls` - File read: `read` , `read many` - File modification: `write` (a whole new file); `edit` (string exact-match replacement); `multi_edit` ; `apply_patch` (applies a structured patch/diff)
Shell execution	Run commands: `bash` , `PowerShell`
IO	`lsp` , git tools like `git_status` , `git_diff` , `git_commit`
External context	MCP tools, Skills
Web search	`web_search` , `web_fetch` , browser tools
Artifacts	Read docs, images; generate HTML, images

学习如何读写和编辑文件系统（通常通过 `bash` 命令）是大语言模型 项基础技能，因此，以文件这种简单形式管理持久记忆，自然也能受益于模型核心能力的提升。

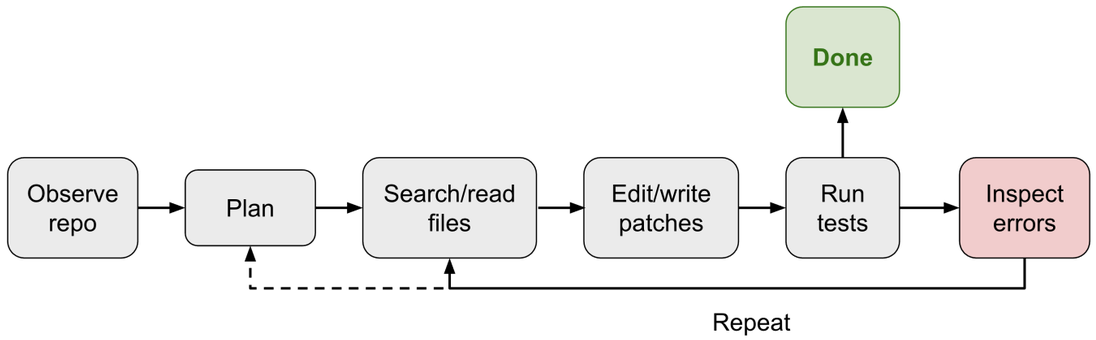
模式 3：子智能体与后台任务

运行框架可以启动多个子智能体并行执行，并监控后台任务。当主智能体需要搜索多个假设、并发运行实验，或委派独立子任务而不污染主上下文时，这种方式很有用。此时，父智能体需要一个小型进程管理器：启动任务、检查日志、取消失败的运行，并将结果合并回主智能体线程。

关键的设计选择是让并行性显式可见且可检查。如果子智能体输出仅存在于临时聊天上下文中，就会很快过时且难以查见。如果将其存储为文件、日志和状态记录，模型就能在中断后恢复，并基于自身的执行历史进行推理。

案例研究：编程智能体运行框架

从 Claude Code、Codex、OpenCode 到 Cursor 风格的智能体，主流编程智能体的核心接口已趋于稳定。它们通常采用如下循环：



借助一组工具，编程智能体能够在给定的代码仓库中进行开发和问题调试，类似于人类开发者使用集成开发环境开展工作。

（此列表并不全面，仅作演示。如有兴趣，请阅读此文。）

分组	工具定义
文件系统	- 文件发现： `glob` , `grep` , `ls` - 文件读取： `read` , `read many` - 文件修改： `write` （一个全新的文件）； `edit` （字符串精确匹配替换）； `multi_edit` ； `apply_patch` （应用结构化补丁/差异）
Shell 执行	运行命令： `bash` , `PowerShell`
IO	`lsp` ，Git 工具，例如 `git_status` , `git_diff` , `git_commit`
外部上下文	MCP 工具，技能
网络搜索	`web_search` , `web_fetch` ，浏览器工具
产物	读取文档、图像；生成 HTML、图像

Group	Tool definitions
Backend processes	Such as: `CronCreate` , `CronDelete` , `CronList`
Agent delegation	Such as: `spawn_agent` , `resume_agent` , `wait_agent` , `list_agents` , `close_agent` , `interrupt_agent` , etc.

Harness Layer vs Core Intelligence?

It is hard to forecast how much the future of RSI will rely on harness engineering, but the near-term path of RSI is unlikely to start as a model directly rewriting its weights. My prediction of a practical near-term path is:

1. Harness engineering will evolve in the direction of meta-methodology (i.e. improving the machinery for getting better answers, not just improving the answer itself). The harness system itself becomes an optimization target, with fewer heuristic rules and more general mechanisms.
2. In turn, mature harnesses enable auto-research for model self-improvement loop and smarter models prevents harnesses from overengineering and keep the system sustainable.

Eventually it is possible that many harness improvements will be *internalized* into core model behavior, but the interface with external context and tools should remain. We have seen a softer version of this pattern with prompt engineering: manual prompt tricks became less central as instruction tuning and model reasoning improved, but *the need to specify goals, constraints, context, and evaluation did not disappear*.

Harness Optimization

The progression in the object being optimized in the harness system is roughly: instruction prompts → structured context → workflow → harness code → optimizer code. As the model becomes more intelligent and powerful, we move toward more complex targets and generic methods.

Context Engineering

Simply appending all the tool responses and model generations into the context can quickly grow out of control as the agentic job horizon increases significantly. Context management is a layer to construct a more structed and concise context for LLM and manage persistant states. There is no doubt that long-context research will keep on making progress but at the moment long-context intelligence and context engineering sometime intertwines.

Agentic Context Engineering (ACE; Zhang et al. 2025) treats context as an evolving playbook rather than an increasingly lengthening prompt. It has three components to maintain one context playbook of bullet points, each with an identifier and a description.

1. *Generator*: produces task trajectories, with reference to bullet points.
2. *Reflector*: distills insights from successful and failed trajectories.
3. *Curator*: updates the structured context with incremental, itemized entries.

分组	工具定义
后端进程	例如： `CronCreate` , `CronDelete` , `CronList`
智能体委派	例如： `spawn_agent` , `resume_agent` , `wait_agent` , `list_agents` , `close_agent` , `interrupt_agent` ，等等。

运行框架层与核心智能?

很难预测递归自我改进的未来会在多大程度上依赖运行框架工程，但近期的递归自我改进路径不太可能从模型直接重写自身权重开始。我预测，近期一条切实可行的路径是：

1. 运行框架工程将朝着元方法论的方向发展（即改进获得更好答案的机制，而不仅仅是改进答案本身）。运行框架系统本身将成为优化目标，启发式规则将更少，通用机制将更多。

2. 反过来，成熟的智能体运行框架能够为模型自我改进循环提供自动化研究能力，而更智能的模型则能防止智能体运行框架过度工程化，使系统保持可持续性。

最终，许多智能体框架改进可能会被内化为核心模型行为，但与外部上下文和工具交互的接口应当保留。我们已经在提示工程中看到了这一模式的温和版本：随着指令微调和模型推理的改进，手动编写提示的技巧不再那么重要，但明确目标、约束、上下文和评估的需求并未消失。

运行框架优化

运行框架系统中优化对象的演进大致为：指令提示词 → 结构化上下文 → 工作流 → 运行框架代码 → 优化器代码。随着模型变得更加智能、能力更强，我们逐步转向更复杂的目标和更通用的方法。

上下文工程

随着智能体任务跨度显著增加，若只是将所有工具响应和模型生成内容追加到上下文中，上下文的规模很快就会失控。上下文管理这一层负责为大语言模型构建更有条理、更简洁的上下文，并管理持久状态。毫无疑问，长上下文研究将持续取得进展，但目前长上下文智能与上下文工程有时会相互交织。

智能体上下文工程（ACE；Zhang 等人，2025年）将上下文视为不断演进的策略手册，而非越来越长的提示词。它通过三个组件维护一本由要点组成的上下文策略手册，每个要点都包含一个标识符和一段描述。

1. 生成器：参考要点生成任务轨迹。2. 反思器：从成功和失败的轨迹中提炼见解。3. 策展器：通过增量式的逐条记录更新结构化上下文。

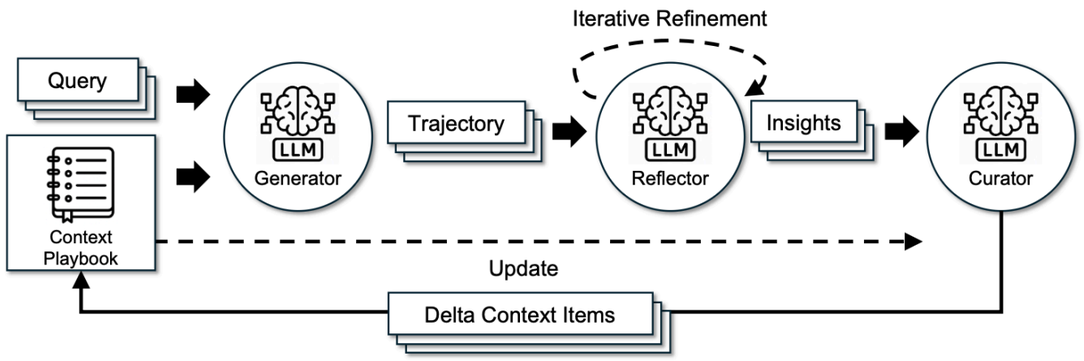


Figure 3: The framework of Agentic Context Engineering (ACE). (Image source: Zhang et al. 2025)

To prevent context collapse and brevity bias during iterative rewrites, one key design choice in ACE is that the curator does not rewrite a full prompt blob. It instead outputs a collection of structured, itemized bullets in the form of (identifier, description), and these bullets are merged into a structured context logbook with deterministic logic. The context items are refined and deduplicated periodically.

The fact that ACE learns insights from rollouts helps us move toward self-managed memory, but the update rules and the overall workflow are still handcrafted. To move toward a more self-improving loop, **Meta Context Engineering** (MCE; Ye et al. 2026) separates the mechanism (how to manage context) from the artifact content (what is in context), running skill evolution at the meta-optimization level and context optimization at the base level.

An MCE skill $s \in \mathcal{S}$ defines a context function $c_s = (\rho_s, F_s)$ and maps an input x to context $c = F_s(x; \rho_s)$, where:

- $\rho_s = \{\rho_1, \ldots, \rho_m\}$ are static components (prompts, knowledge bases, code libraries).
- $F_s = \{F_1, \ldots, F_k\}$ are dynamic operators (search, selection, filtering, formatting).

The bi-level optimization is to find the best context c_s^* given skill s on the training data, while the outer loop finds the optimal skill that provides the best performance on the validation set:

$$\textbf{Inner: } c_s^* = \arg\max_{c_s} J_{\text{train}}(c_s; s) \quad \textbf{Outer: } s^* = \arg\max_{s \in \mathcal{S}} J_{\text{val}}(c_s^*)$$

The skill database tracks the history of previous skills, context functions and eval metrics $\mathcal{H}_{k-1} = \{(s_i, c_i, J_i^{\text{train}}, J_i^{\text{val}})\}_{i=1}^{k-1}$. A meta-level agent performs agentic crossover over prior skills to create a new skill given a task τ: $s_k = \textbf{crossover}(\tau, \mathcal{H}_{k-1})$.

Then a base-level context engineer executes the skill s_k and learns the context function from rollout feedback $\mathcal{R}_k$, guided by the current skill: $c_k = \textbf{engineer}(\tau, s_k; c_{k-1}^*, \mathcal{R}_k)$.

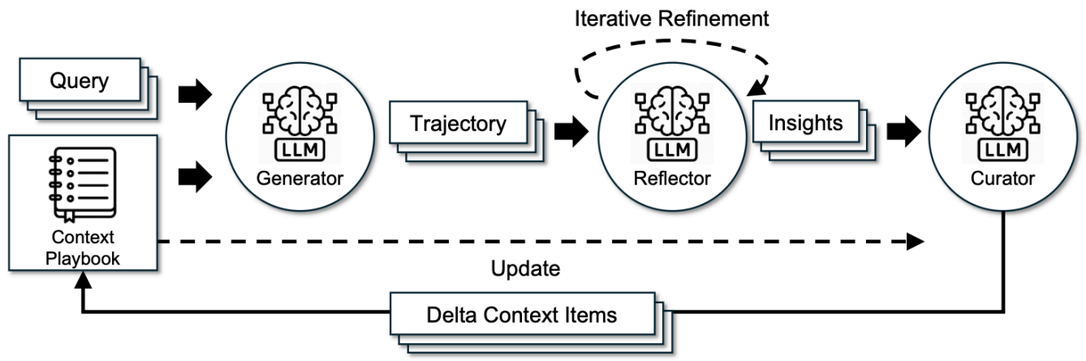


图3：智能体上下文工程（ACE）的框架。（图片来源：Zhang 等人，2025年）

为防止迭代重写过程中出现上下文坍塌和简短偏差，ACE 的一项关键设计选择是让策展器不再重写整个提示词块，而是输出一组以（标识符，描述）形式组织的结构化条目，再通过确定性逻辑将这些条目合并到结构化上下文日志中。上下文条目会定期进行优化和去重。

ACE 能够从轨迹采样中学习洞见，这有助于我们迈向自管理记忆，但更新规则和整体工作流仍由人工设计。为了迈向更完善的自我改进循环，**元上下文工程**（MCE；Ye 等人，2026年）将机制（如何管理上下文）与产物内容（上下文中包含什么）分离，在元优化层面进行技能演化，在基础层面进行上下文优化。

MCE技能定义了一个上下文函数，并将输入映射为上下文，其中：

- $\rho_s = \{\rho_1, \ldots, \rho_m\}$ 为静态组件（提示词、知识库、代码库）。为动态算子（搜索、选
- $F_s = \{F_1, \ldots, F_k\}$ 择、过滤、格式化）。

双层优化旨在训练数据上寻找给定技能下的最佳上下文，而外层循环则寻找能在验证集上取得最佳表现的最优技能：

$$\textbf{Inner: } c_s^* = \arg\max_{c_s} J_{\text{train}}(c_s; s) \quad \textbf{Outer: } s^* = \arg\max_{s \in \mathcal{S}} J_{\text{val}}(c_s^*)$$

技能数据库跟踪以往技能、上下文函数和评估指标的历史记录。元层智能体对以往技能执行智能体式交叉操作，以针对给定任务创建新技能：。

然后，基础层上下文工程师执行该技能，并在当前技能的指导下，从轨迹采样反馈中学习上下文函数：。

k 1

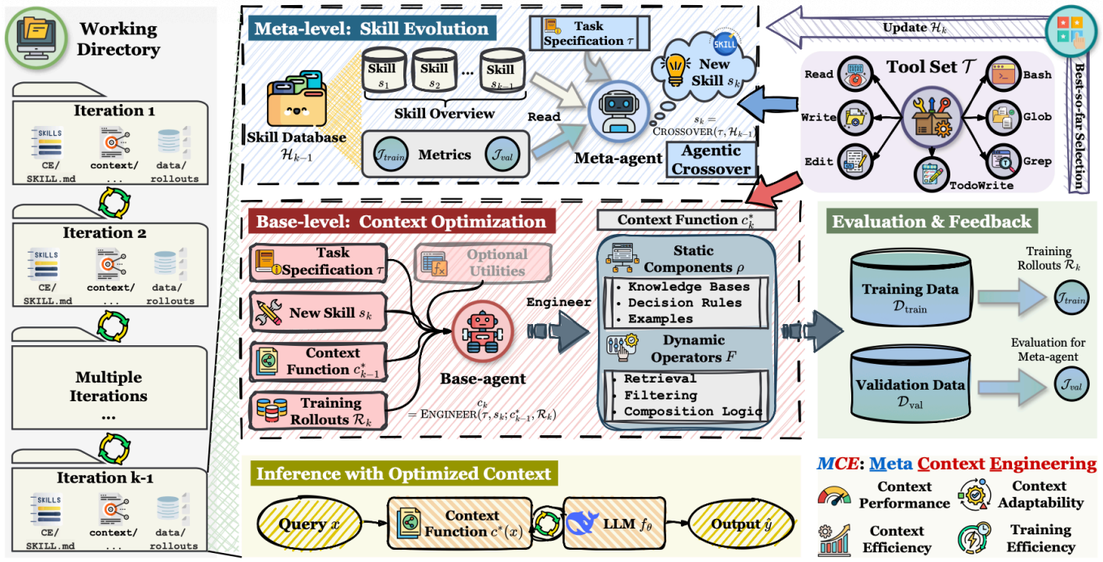


Figure 4: The framework of Meta Context Engineering (MCE): meta-level skill evolution searches over context-management mechanisms, while the base level optimizes the task context. (Image source: Ye et al. 2026)

MCE does not enforce a heuristic rule for how to structure context as ACE does. It uses *free-form skills* to store the most important knowledge for a task, and evolves the skill and the skill-conditioned context iteratively together.

Implementation-wise, a context function $\boldsymbol{c}$ is instantiated as a collection of files in a dedicated directory, including both static (`skill.md`) and dynamic (context and data rollouts) components. Both meta-level and base-level optimization are executed in agentic coding envs with a standard tool set,

$$\mathcal{T} = \{\texttt{Read}, \texttt{Write}, \texttt{Edit}, \texttt{Bash}, \texttt{Glob}, \texttt{Grep}, \texttt{TodoWrite}\}$$

Meta-Harness (Lee et al. 2026) moves another level deeper: the optimized object is the *code* that determines and optimizes what information should be stored, retrieved, and presented to the model. "Meta-" in its name means it is a harness for optimizing harnesses.

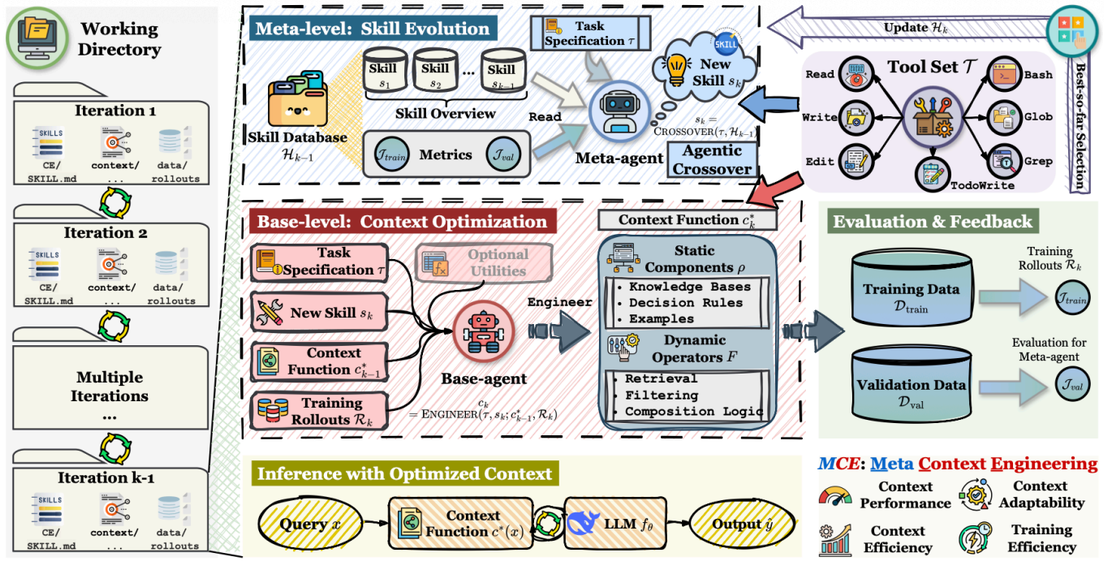


图4：元上下文工程（MCE）框架：元层级技能演化在上下文管理机制中进行搜索，而基础层级则优化任务上下文。（图片来源：Ye 等人，2026年）

MCE 不像 ACE 那样强制采用启发式规则来组织上下文。它使用自由形式技能来存储任务中最重要的知识，并通过迭代共同演进技能及以技能为条件的上下文。在实现层面，上下文函数被实例化为专用目录中的一组文件，其中既包含静态组件（ `skill.md` ），也包含动态组件（上下文和数据展开轨迹）。元层级优化和基础层级优化都在配备标准工具集的智能体编程环境中执行，

$$\mathcal{T} = \{\texttt{Read}, \texttt{Write}, \texttt{Edit}, \texttt{Bash}, \texttt{Glob}, \texttt{Grep}, \texttt{TodoWrite}\}$$

Meta-Harness（Lee 等人，2026年）又深入了一个层次：其优化对象是代码，这些代码决定并优化应存储、检索哪些信息，以及应向模型呈现哪些信息。名称中的"Meta-"表示，它是一个用于优化智能体运行框架的运行框架。

Algorithm 1 Meta-Harness outer loop over harnesses

Input: tasks $\mathcal{X}$, LLM M, proposer P, iterations N
Initialize: population $\mathcal{H}$ ▷ Initial set of valid harnesses
Initialize: filesystem $\mathcal{D} \leftarrow \emptyset$ ▷ stores code, scores, traces
for $H \in \mathcal{H}$ **do**
    $E_H \leftarrow \text{Evaluate}(H, M, \mathcal{X})$
    $\mathcal{D} \leftarrow \mathcal{D} \cup \{(H, E_H)\}$
for $t = 1 \dots N$ **do**
    Proposer P queries filesystem $\mathcal{D}$ ▷ inspects prior harnesses and scores
    Proposer P proposes k new harnesses $\{H_1, \dots, H_k\}$
    for H in $\{H_1, \dots, H_k\}$ **do**
        if H passes interface validation **then**
            $\mathcal{D} \leftarrow \mathcal{D} \cup \{(H, \text{EVALUATE}(H, M, \mathcal{X}))\}$
return Pareto frontier of harnesses stored in $\mathcal{D}$

Figure 5: The Meta-Harness outer-loop optimization algorithm. (Image source: Lee et al. 2026)

The proposer for creating a new harness is itself a coding agent and the final output is a collection of harness candidates on the Pareto frontier.

- The entire execution history is accessible via a file system, and thus the coding agent uses commands like `grep` or `cat` to read through it instead of shoveling everything into a single prompt context.
- The proposed harness is a dictionary in the file system containing its own source code, scores, rollout trajectories, and state updates.
- The mete-harness loop iteratively creates new harnesses, and only qualified ones are kept.

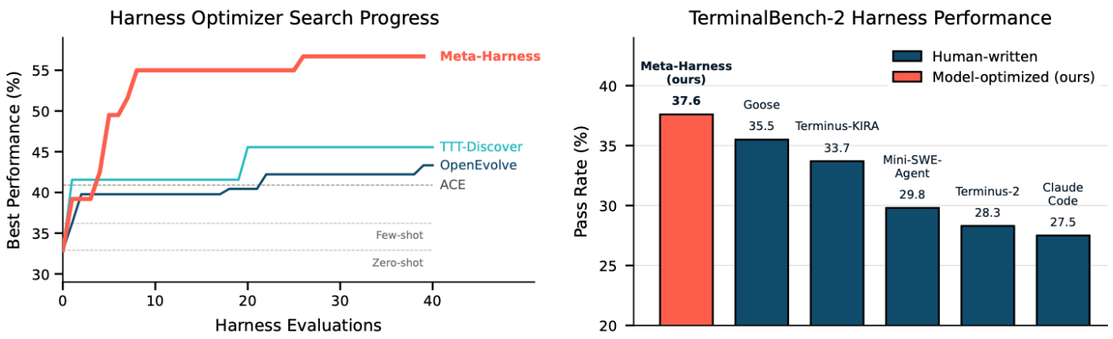


Figure 6: The performance of Meta-Harness on (Left) text classification with a small number of iterations and (Right) TerminalBench-2. Note that the search in the TerminalBench-2 experiment is initialized from Terminus-KIRA and Terminus-2, two very strong harnesses. (Image source: Lee et al. 2026)

Still, the important lesson is clear: once harness design becomes an executable search space, a strong coding agent can exploit the same design space human engineers use.

Algorithm 1 Meta-Harness outer loop over harnesses

Input: tasks $\mathcal{X}$, LLM M, proposer P, iterations N
Initialize: population $\mathcal{H}$ ▷ Initial set of valid harnesses
Initialize: filesystem $\mathcal{D} \leftarrow \emptyset$ ▷ stores code, scores, traces
for $H \in \mathcal{H}$ **do**
    $E_H \leftarrow \text{Evaluate}(H, M, \mathcal{X})$
    $\mathcal{D} \leftarrow \mathcal{D} \cup \{(H, E_H)\}$
for $t = 1 \dots N$ **do**
    Proposer P queries filesystem $\mathcal{D}$ ▷ inspects prior harnesses and scores
    Proposer P proposes k new harnesses $\{H_1, \dots, H_k\}$
    for H in $\{H_1, \dots, H_k\}$ **do**
        if H passes interface validation **then**
            $\mathcal{D} \leftarrow \mathcal{D} \cup \{(H, \text{EVALUATE}(H, M, \mathcal{X}))\}$
return Pareto frontier of harnesses stored in $\mathcal{D}$

图 5：Meta-Harness 外循环优化算法。（图片来源：Lee 等人，2026年）

用于创建新运行框架的提议器本身就是一个编程智能体，最终输出则是帕累托前沿上的一组候选运行框架。

- 整个执行历史都可以通过文件系统访问，因此编程智能体会使用诸如以下的命令 e `grep` `cat` 来通读执行历史，而不是将所有内容一股脑地塞进单个提示上下文中。所提出的运
- 行框架是文件系统中的一个字典，其中包含其自身的源代码、评分、展开轨迹和状态更新。
  Meta-Harness 循环通过迭代创建新的智能体运行框架，并仅保留符合条件的框架。
-

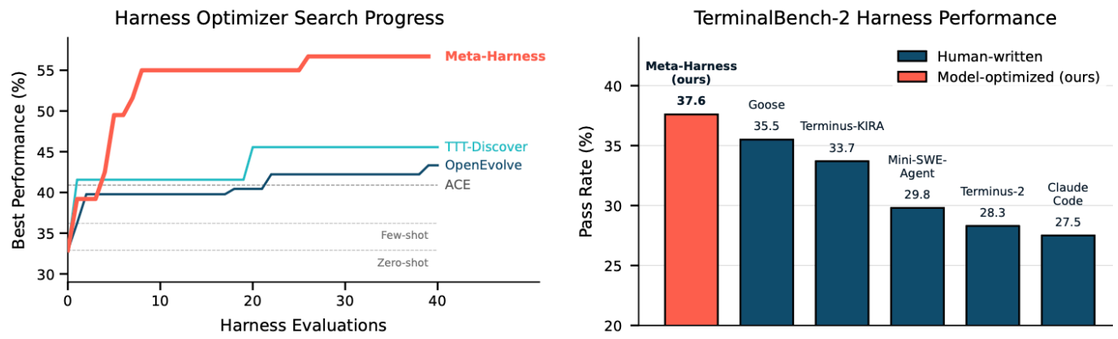


图 6：Meta-Harness 在少量迭代下执行文本分类任务（左）以及在 TerminalBench-2 上（右）的性能。请注意，TerminalBench-2 实验中的搜索以 Terminus-KIRA 和 Terminus-2 这两个非常强大的智能体运行框架为起点。（图片来源：Lee 等人，2026年）

不过，重要的启示已经很明确：一旦智能体运行框架设计成为可执行搜索空间，强大的编程智能体就能利用与人类工程师相同的设计空间。

Workflow Design

Workflow design in harness engineering can be handcrafted by domain experts. Taking auto-research as an example, various frameworks have been proposed and tested. The **AI Scientist** system (Lu et al. 2026) builds a pipeline to propose research ideas, write code, run experiments, analyze results, write a manuscript, and perform peer review. Meng et al. (2026) make verifiability the central design constraint in **ScientistOne**, where every claim (citation, numerical, methodological, conclusion) must trace to an evidence source and is audited by Chain-of-Evidence checks.

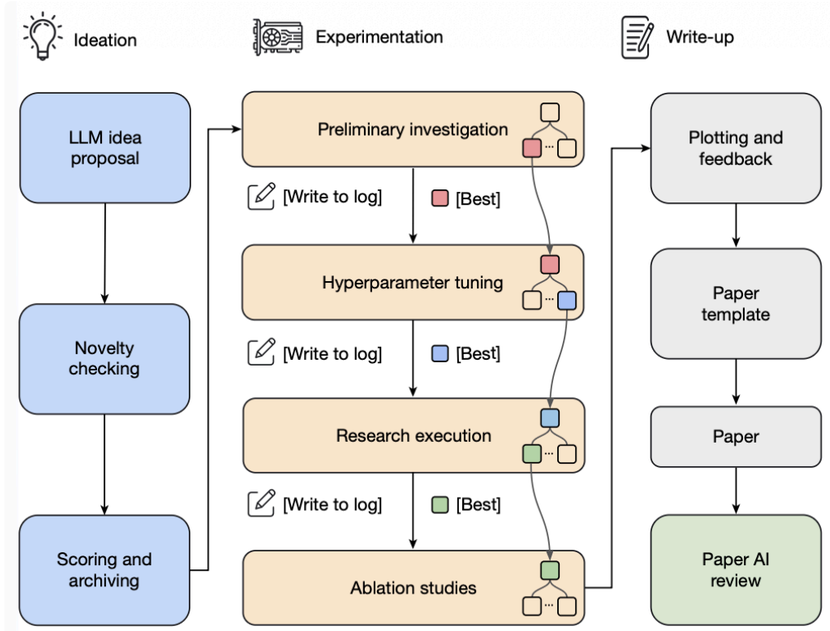


Figure 7: AI Scientist pipeline for idea generation, experimentation, paper writing, and review. (Image source: Lu et al. 2026)

The **Autodata** agent (Kulikov et al. 2026) is designed to work as a data scientist for generating training and evaluation data. The main agent manages a *challenger* that proposes problems, a *weak solver*, a *strong solver*, and a *verifier/judge*, aiming to synthesize data at the "just right" level of difficulty, meaning that the strong solver succeeds but the weak solver fails.

In Autodata, the challenger prompt is updated iteratively according to feedback from the solvers and verifier. The limitation here is that synthesized tasks are used to fine-tune weak solvers but not strong solvers; if the loop cannot iteratively improve the strong model, it is more like indirect distillation over a generated prompt distribution, with less RSI flavor.

工作流设计

运行框架工程中的工作流设计可以由领域专家手工完成。以自动化研究为例，目前已有多种框架被提出并经过测试。**AI Scientist** 系统（Lu 等人，2026年）构建了一条流水线，用于提出研究构想、编写代码、运行实验、分析结果、撰写稿件以及开展同行评审。Meng 等人（2026年）将可验证性作为 **ScientistOne** 的核心设计约束，其中每项主张（引文、数值、方法、结论）都必须能够追溯至证据来源，并接受证据链检查的审查。

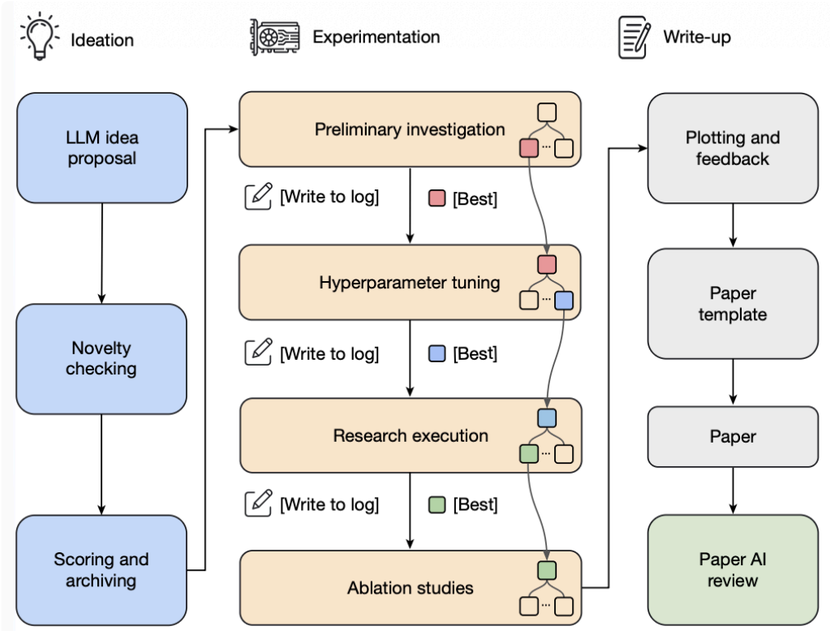


图 7：用于构想生成、实验开展、论文写作和评审的 AI Scientist 流水线。（图片来源：Lu 等人，2026年）

该 **Autodata** 智能体（Kulikov 等人，2026年）旨在作为数据科学家，生成训练与评估数据。主智能体管理着一个挑战者，由其提出问题，同时还管理着一个弱求解器、一个强求解器以及一个验证器/评判器，目标是合成难度“恰到好处”的数据，即强求解器能够成功求解，而弱求解器无法求解。

在 Autodata 中，挑战者提示词会根据求解器和验证器的反馈进行迭代更新。这里的局限在于，合成任务被用于微调弱求解器，而非强求解器；如果这一循环无法迭代改进强模型，那么它就更像是在生成的提示词分布上进行间接蒸馏，递归自我改进的意味则较淡。

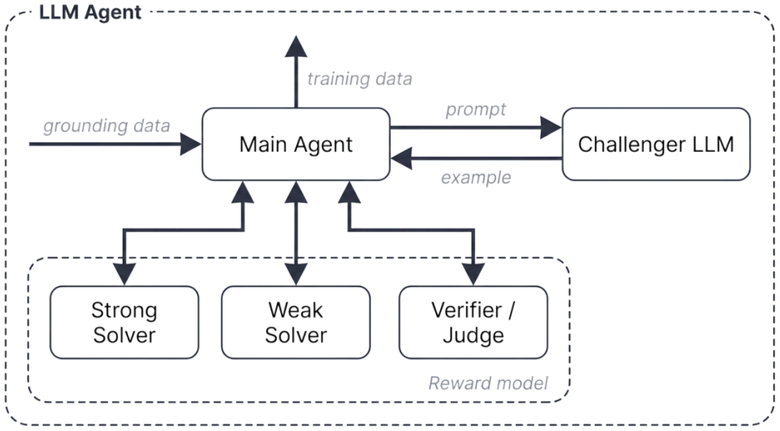


Figure 8: Autodata agentic workflow design for generating synthetic training and evaluation data around challenger, solver, and verifier roles. (Image source: Kulikov et al. 2026)

The design space for workflow is *enormous*, and naturally we can think of workflow design as a search problem, and therefore we should be able to find good solutions by algorithms rather than only manually craft them. Following this direction, **Automated Design of Agentic Systems** (ADAS; Hu et al. 2025) formulates agent design itself as an optimization problem, "meta-agent search" where a meta-agent proposes new designs of agentic workflows.

1. Initialize an archive of agentic workflows with simple agents such as CoT and self-refine.
2. Ask a meta-agent to program new agents, all in *code*, inspired by existing solutions in the archive.
    - The meta-agent first generates a high-level description of the new workflow, and then implements it in code.
    - The draft program then goes through two self-refine steps (i.e. ask the model to provide feedback and then ask the same model to refine the previously generated outputs based on the feedback; Madaan et al. 2023) by the meta-agent to check its novelty.
3. Evaluate each new candidate and add successful ones back to the archive.
4. Repeat steps 2-3 until the maximum iteration count is reached.

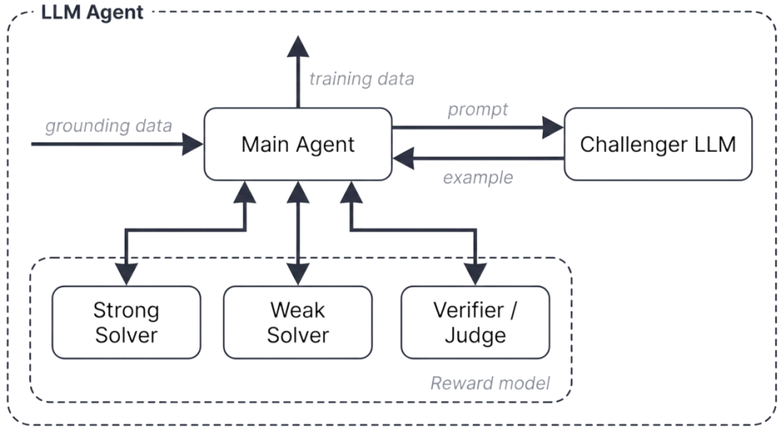


图8：Autodata 智能体工作流设计，围绕挑战者、求解者和验证器角色生成合成训练与评估数据。（图片来源：Kulikov 等人，2026年）

工作流的设计空间极其庞大，因此我们自然可以将工作流设计视为一个搜索问题，并且应该能够通过算法找到好的解决方案，而不只是依靠手工设计。沿着这一方向，**智能体系统自动化设计**（ADAS；Hu 等人，2025年）将智能体设计本身表述为一个优化问题，即“元智能体搜索”，由元智能体提出新的智能体工作流设计。

1. 使用思维链和 Self-Refine 等简单智能体，初始化一个智能体工作流档案库。
2. 让元智能体以档案库中的现有解决方案为灵感，完全用代码编写新的智能体。
    - 元智能体首先生成新工作流的高层描述，然后用代码实现。随后，元智能体对程序草稿执行两个
    - Self-Refine 步骤（即先让模型提供反馈，再让同一模型根据反馈改进先前生成的输出；Madaan 等人，2023年），以检查其新颖性。
3. 评估每个新候选方案，并将成功的方案添加回档案库。
4. 重复步骤 2-3，直至达到最大迭代次数。

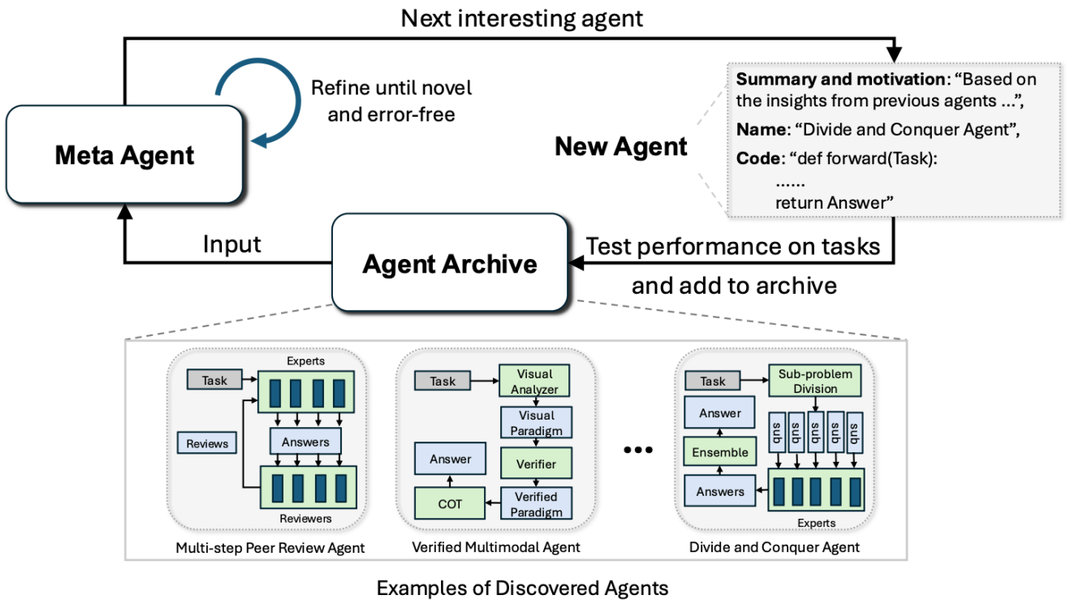


Figure 9: Illustration of Automated Design of Agentic Systems (ADAS). (Image source: Hu et al. 2025)

AFlow (Zhang et al. 2025) represents an agentic workflow as a graph, where nodes represent LLM-invoking actions and edges implement logical operations in code. The workflow optimization relies on MCTS (Monte Carlo Tree Search):

1. Initialize the starting workflow W_0 in the tree with a template.
2. Select a workflow node using a soft mixture of score and uniform exploration.
3. Expand it by asking an LLM to produce a modified workflow conditioned on its evaluation performance.
4. Execute and evaluate the new workflow.
5. Add it back to the tree if the new workflow shows improvement within a budget of N rounds.
6. Repeat steps 2-5 and stop when the top-k average score plateaus or hit the budget.

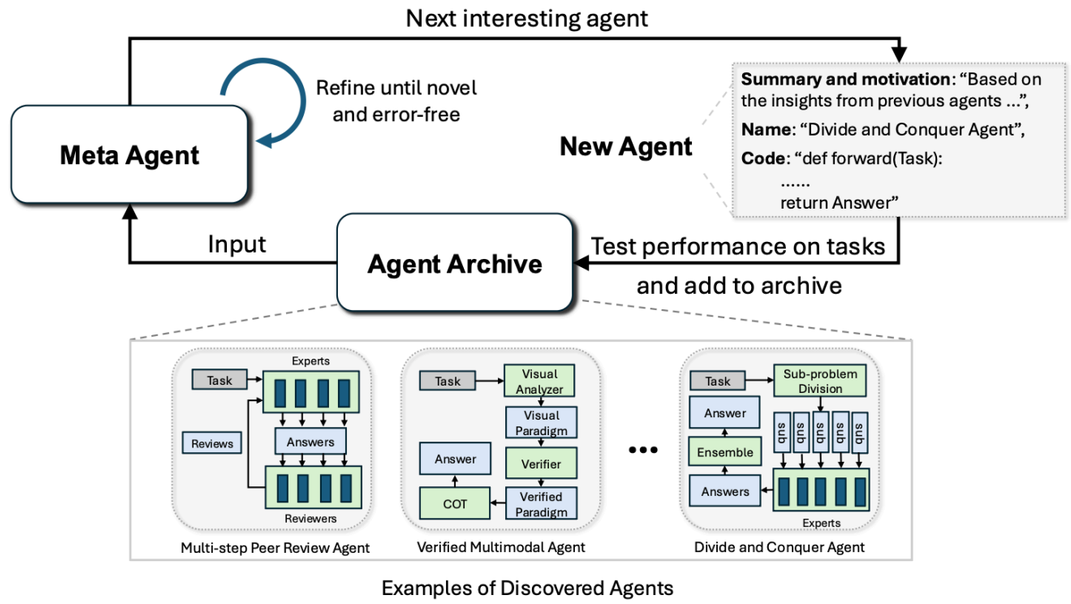


图 9：智能体系统自动化设计（ADAS）示意图。（图片来源：Hu 等人，2025年）

AFlow（Zhang 等人，2025年）将智能体工作流表示为图，其中节点表示大语言模型调用动作，边则通过代码实现逻辑运算。工作流优化依赖于 MCTS（蒙特卡洛树搜索）：

1. 使用模板初始化树中的起始工作流。2. 采用评分与均匀探索的软混合策略选择一个工作流节点。3. 让大语言模型根据该工作流的评估表现生成修改后的工作流，以此扩展该节点。4. 执行并评估新工作流。5. 如果新工作流在限定轮数内表现出改进，则将其添加回树中。6. 重复步骤 2–5，直到排名靠前的工作流的平均分趋于稳定，或达到轮数上限。

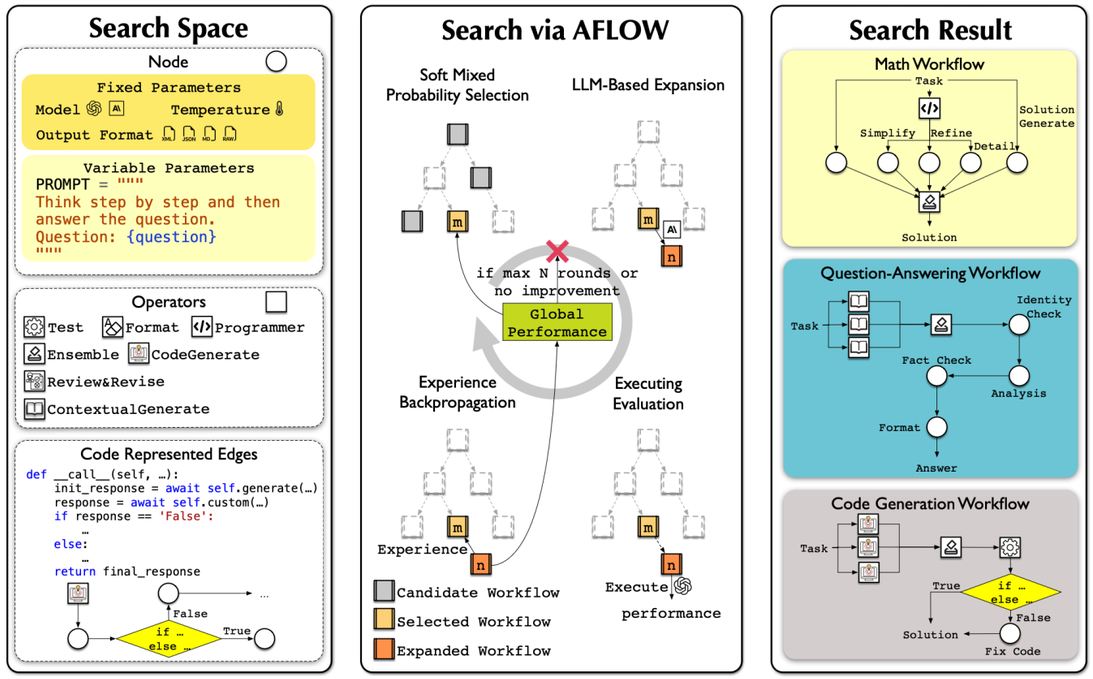


Figure 10: AFlow optimization process over a tree of workflow candidates. (Image source: Zhang et al. 2025)

图10：AFlow 在候选工作流树上的优化过程。（图片来源：Zhang 等人，2025年）

Experiments of AFlow in QA, code, and math tasks showed decent improvement of AFlow over manually designed workflows and ADAS.

Method	Benchmarks						Avg.
	HotpotQA	DROP	HumanEval	MBPP	GSM8K	MATH	
IO (GPT-4o-mini)	68.1	68.3	87.0	71.8	92.7	48.6	72.8
CoT (Wei et al., 2022)	67.9	78.5	88.6	71.8	92.4	48.8	74.7
CoT SC (5-shot) (Wang et al., 2022)	68.9	78.8	91.6	73.6	92.7	50.4	76.0
MedPrompt (Nori et al., 2023)	68.3	78.0	91.6	73.6	90.0	50.0	75.3
MultiPersona (Wang et al., 2024a)	69.2	74.4	89.3	73.6	92.8	50.8	75.1
Self Refine (Madaan et al., 2023)	60.8	70.2	87.8	69.8	89.6	46.1	70.7
ADAS (Hu et al., 2024)	64.5	76.6	82.4	53.4	90.8	35.4	67.2
Ours (AFlow)	**73.5**	**80.6**	**94.7**	**83.4**	**93.5**	**56.2**	**80.3**

Figure 11: AFlow experiments in comparison to manual methods and ADAS. (Image source: Zhang et al. 2025)

在问答、代码和数学任务上的实验表明，与人工设计的工作流和 ADAS 相比，AFlow 取得了较为显著的提升。

Method	Benchmarks						Avg.
	HotpotQA	DROP	HumanEval	MBPP	GSM8K	MATH	
IO (GPT-4o-mini)	68.1	68.3	87.0	71.8	92.7	48.6	72.8
CoT (Wei et al., 2022)	67.9	78.5	88.6	71.8	92.4	48.8	74.7
CoT SC (5-shot) (Wang et al., 2022)	68.9	78.8	91.6	73.6	92.7	50.4	76.0
MedPrompt (Nori et al., 2023)	68.3	78.0	91.6	73.6	90.0	50.0	75.3
MultiPersona (Wang et al., 2024a)	69.2	74.4	89.3	73.6	92.8	50.8	75.1
Self Refine (Madaan et al., 2023)	60.8	70.2	87.8	69.8	89.6	46.1	70.7
ADAS (Hu et al., 2024)	64.5	76.6	82.4	53.4	90.8	35.4	67.2
Ours (AFlow)	**73.5**	**80.6**	**94.7**	**83.4**	**93.5**	**56.2**	**80.3**

Self-Improving Harness

Either context engineering or workflow design is only one part of a harness. We need to search through the entire design space and optimize context-management logic, workflow, permissions, and many other harness components together. As we have seen in work like Meta-Harness, ADAS, and AFlow, ✨**code**✨ is a **universal**

自我改进的智能体运行框架

无论是上下文工程还是工作流设计，都只是运行框架的一部分。我们需要搜索整个设计空间，并对上下文管理逻辑、工作流、权限以及许多其他框架组件进行联合优化。正如我们在 Meta-Harness、ADAS 和 AFlow 等工作中所见，✨**代码**✨ 是一种**通用的**

language for defining programs and systems. In simple words, a harness is code that programs how prompts, tool calls, subagents, control flow, memory, and workflow logic work together. If an LLM can optimize the code that executes agents, it can access a *much larger design space* than hand-written prompts.

Self-Taught Optimizer (STOP; Zelikman et al. 2023) is one of the early examples of recursive scaffolding improvement. A seed improver I_0 at step $t = 0$ takes an initial solution s, a utility function u, and a black-box language model M, and returns an improved solution s', that is, $s' = I(u, s; M)$. The goal of STOP is not directly to improve s but *to improve the improver I itself*.

First, let's define the meta-utility as the average utility of a given improver function I over a collection of downstream tasks $\mathcal{D}$:

$$\hat{u}(I) \triangleq \frac{1}{|\mathcal{D}|}\mathbb{E}_{(u,s)\sim\mathcal{D}}[u(I(u, s; M))]$$

Because improving the improver function is an optimization problem itself, we can recursively get a new version of I_t based on I_{t-1}'s performance measured by meta-utility via a self-improvement update:

$$I_t = I_{t-1}(\hat{u}, I_{t-1}; M)$$

Algorithm 1: Self-Taught Optimizer (STOP)

Input: Seed improver I_0, language model L, recursion depth T, downstream tasks D
Output: An improved improver I_T
for $t = 1$ **to** T **do**
  $I_t \leftarrow I_{t-1}(\hat{u}, I_{t-1}, L)$ `// Update improver based on meta-utility` $\hat{u}$
return I_T `// Return the final improver`

Function $\tilde{u}(I)$**:**
  utility_sum $\leftarrow 0$ `// Maintain sum of downstream task utilities`
  for $(u, S) \in D$ **do**
    $S' \leftarrow I(u, S, L)$ `// Improve initial solution` S `using improver` I
    utility_sum += $u(S')$ `// Add new utility`
  return utility_sum$/|D|$ `// Return expected utility`

Figure 12: Algorithm of Self-Taught Optimizer (STOP). (Image source: Zelikman et al. 2023)

In their experiments, the improved improver discovered various strategies, such as genetic algorithms, decomposing and improving parts, multi-armed prompt bandits, simulated annealing, varying temperature, and beam/tree search. This is analogous to how a harness workflow can be represented as an object for optimization.

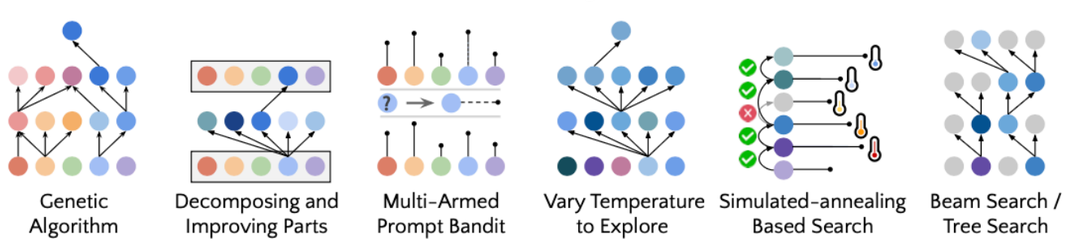


Figure 13: Examples of self-improvement strategies discovered by STOP. (Image source: Zelikman et al. 2023)

语言，用于定义程序和系统。简单来说，运行框架就是通过编程来规定提示词、工具调用、子智能体、控制流、记忆和工作流逻辑如何协同工作的代码。如果大语言模型能够优化运行智能体的代码，它就能探索比手工编写的提示词大得多的设计空间。

自学优化器（STOP；Zelikman 等人，2023年）是递归式脚手架改进的早期示例之一。某一步的种子改进器接收一个初始解、一个效用函数和一个黑盒语言模型，并返回一个改进后的解，即 。STOP 的目标并非直接进行改进，而是改进改进器本身。

首先，我们将元效用定义为给定改进器函数在一组下游任务上的平均效用：

$$\hat{u}(I) \triangleq \frac{1}{|\mathcal{D}|}\mathbb{E}_{(u,s)\sim\mathcal{D}}[u(I(u, s; M))]$$

由于改进改进器函数本身就是一个优化问题，因此我们可以根据元效用衡量的表现，通过自我改进更新，递归地获得新版本：

$$I_t = I_{t-1}(\hat{u}, I_{t-1}; M)$$

Algorithm 1: Self-Taught Optimizer (STOP)

Input: Seed improver I_0, language model L, recursion depth T, downstream tasks D
Output: An improved improver I_T
for $t = 1$ **to** T **do**
  $I_t \leftarrow I_{t-1}(\hat{u}, I_{t-1}, L)$ `// Update improver based on meta-utility` $\hat{u}$
return I_T `// Return the final improver`

Function $\tilde{u}(I)$**:**
  utility_sum $\leftarrow 0$ `// Maintain sum of downstream task utilities`
  for $(u, S) \in D$ **do**
    $S' \leftarrow I(u, S, L)$ `// Improve initial solution` S `using improver` I
    utility_sum += $u(S')$ `// Add new utility`
  return utility_sum$/|D|$ `// Return expected utility`

图 12：自学优化器（STOP）的算法。（图片来源：Zelikman 等人，2023年）

在他们的实验中，经过改进的改进器发现了多种策略，例如遗传算法、对各部分进行分解和改进、多臂提示词老虎机、模拟退火、调整温度，以及束搜索／树搜索。这类似于将编排框架工作流表示为可供优化的对象。

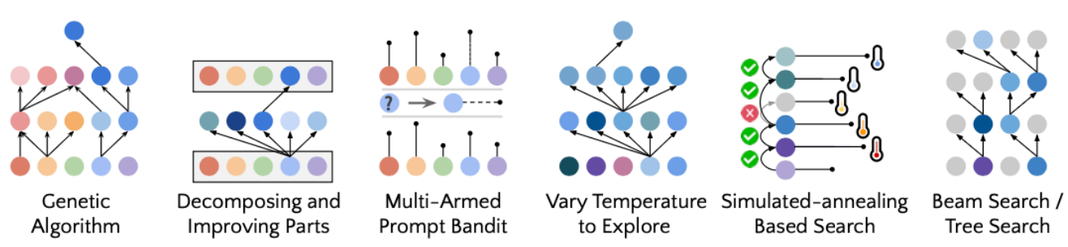


图 13：STOP 发现的自我改进策略示例。（图片来源：Zelikman 等人，2023年）

A *cautionary* result in Zelikman et al. (2023)'s findings is that STOP improved mean downstream performance across iterations with GPT-4 but degraded with weaker models like GPT-3.5 and Mixtral. Recursive structure alone is not enough. The base model must be *capable enough* to improve the mechanism. This implies that harness improvement enables better deployment of the model but intelligence is still the core.

Lin et al. (2026) investigated the dependency of harness evolution on model capabilities in more details. They disentangled two axes: (1) *harness-updating* refers to the capability of producing useful harness edits and (2) *harness-benefit* denotes the capability of utilizing the updated harness, to achieve better task solving. Interestingly a range of model of different sizes and core intelligence, from Qwen3.5-9B to Claude Opus 4.6, were observed in their experiments to show similar harness updating capability; the 9B harness proposer/evolver is able to write a skill procedurally isomorphic to Opus. To best utilize a harness, a model needs to invoke skills/tools correctly and timely and be good at long-horizon instruction following.

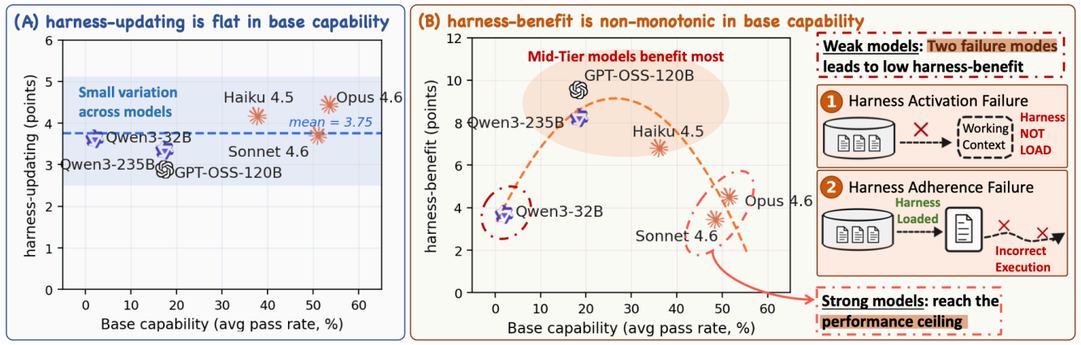


Figure 14: Main results: (A) harness updating capability is measured flat across a range of models from Qwen2-32B to Opus 4.6; (B) harness benefit capability is non-monotonic where middle tier models benefit the most. (Image source: Lin et al. 2026)

A more recent work, **Self-Harness** (Zhang et al. 2026), relies on LLM agents to improve their own harness via a propose-evaluate-accept loop.

Zelikman 等人（2023年）的研究发现中，有一项值得警惕的结果：使用 GPT-4 时，STOP 的平均下游性能随着迭代而提升，但使用 GPT-3.5 和 Mixtral 等较弱模型时，性能反而下降。仅有递归结构还不够。基础模型必须具备足够的能力，才能改进这一机制。这意味着，智能体框架改进有助于更好地部署模型，但智能仍然是核心。

Lin 等人（2026年）更详细地研究了运行框架演化对模型能力的依赖。他们区分了两个维度：（1）智能体运行框架更新指生成有用的智能体运行框架编辑的能力；（2）智能体运行框架受益指利用更新后的运行框架更好地解决任务的能力。有趣的是，他们在实验中观察到，从 Qwen3.5-9B 到 Claude Opus 4.6，一系列规模和核心智能各不相同的模型展现出了相近的智能体运行框架更新能力；9B 智能体运行框架提议器/演化器能够编写出在流程上与 Opus 所编写的技能同构的技能。要充分利用运行框架，模型需要正确、及时地调用技能/工具，并擅长长程指令遵循。

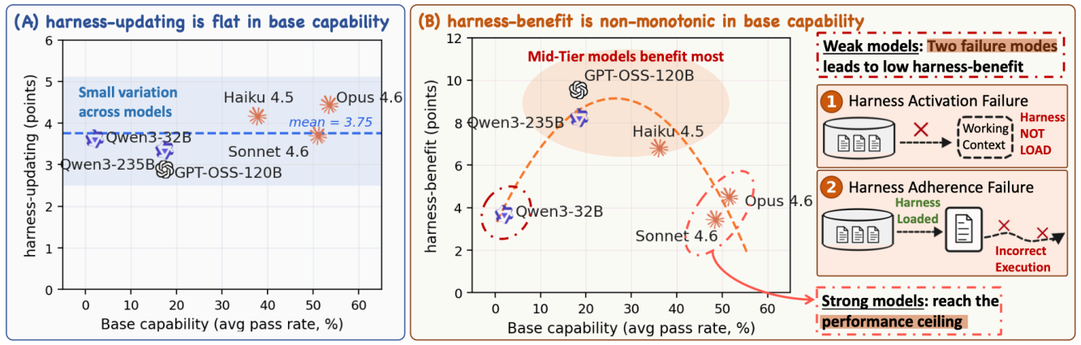


图14：主要结果：(A) 从 Qwen2-32B 到 Opus 4.6，一系列模型的智能体运行框架更新能力测量结果基本持平；(B) 智能体运行框架受益能力呈非单调变化，其中中等水平模型获益最多。（图片来源：Lin 等人，2026年）

一项较新的研究 **Self-Harness**（Zhang 等人，2026年）依靠大语言模型智能体，通过提议—评估—接受循环改进自身的运行框架。

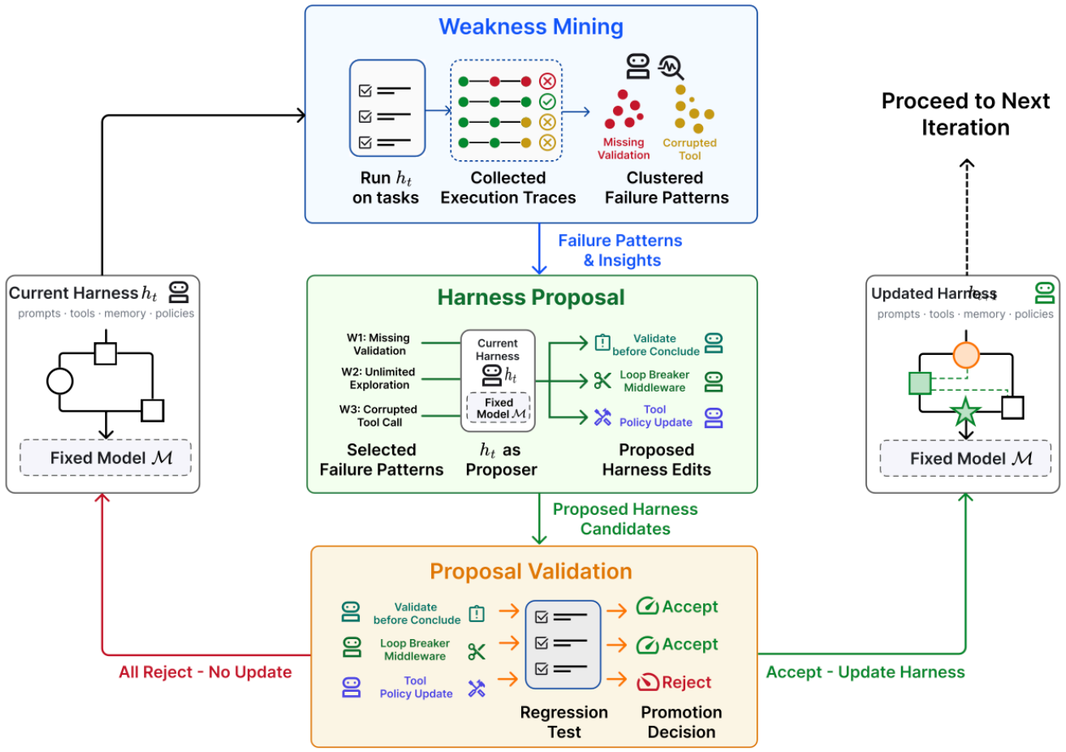


Figure 15: Self-Harness uses a loop of weakness mining, bounded harness proposal, and validation to update a harness. (Image source: Zhang et al. 2026)

The loop in Self-Harness has three stages:

1. *Weakness mining*: cluster failures into verifier-grounded failure patterns.
    - The current harness $\boldsymbol{h_t}$ is used to evaluate on tasks and execution traces are collected for analysis.
    - Note that two runs can share the same verifier outcome in the error logs on the surface, such as timeout or missing artifact, while having different causal mechanisms. Therefore we need a failure record of rich information, containing the terminal verifier-level cause, the causal status of the relevant agent behavior, and the abstract agent mechanism exposed by the trace, to uncover the root causes.
2. *Harness proposal*: propose bounded harness edits based on mined failure patterns.
    - The same model is invoked under $\boldsymbol{h_t}$ as a proposer.
    - The model is provided with a bounded proposal context: (1) the editable surfaces of the current harness, (2) the verifier-grounded failure patterns from the evaluation system, (3) records of passing behaviors that should be preserved, and (4) summaries of previously attempted edits.
    - Harness edits should prefer recurrent error patterns that are addressable (e.g. not task-specific difficulty) and can be resolved by narrow changes.
    - Harness edit candidates should be distinct and diverse.
3. *Proposal validation*: validate and merge qualified edits to create a new harness $\boldsymbol{h_{t+1}}$.

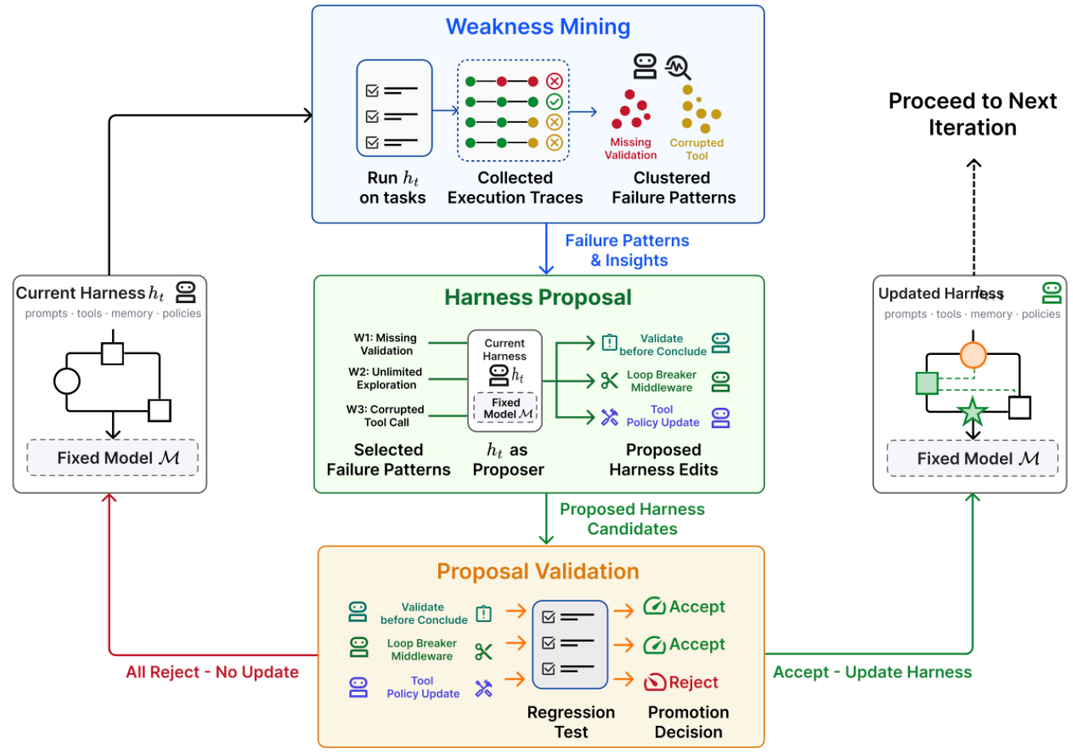


图15：Self-Harness 通过弱点挖掘、有界执行框架提案和验证构成的循环来更新运行框架。（图片来源：Zhang 等人，2026年）

Self-Harness 的循环包含三个阶段：

1. 弱点挖掘：将失败聚类为基于验证器的失败模式。
    - 使用当前运行框架对任务进行评估，并收集执行轨迹以供分析。请注意，两次运行在错误日志中表面上可能
    - 具有相同的验证器结果，例如超时或产物缺失，但其因果机制可能不同。因此，我们需要一份信息丰富的失败记录，其中包含最终验证器层级原因、相关智能体行为的因果状态，以及追踪记录所揭示的抽象智能体机制，以揭示根本原因。
2. 执行框架提案：根据挖掘出的失败模式，提出有界执行框架修改。
    - 调用同一模型作为提议器。为模型提供有界提案上下文，包括：(1) 当前运行框架的可编辑范围，(2) 来自评
    - 估系统的基于验证器的失败模式，(3) 应保留的通过验证的行为记录，以及 (4) 此前尝试过的修改摘要。智能体运行框架编辑应优先针对可处理（例如，并非任务特定难度所致）且能通过小范围改动解决的重复出现的错误模式。执行框架修改候选项应彼此有别且具有多样性。
    -
    -
3. 提案验证：验证并合并符合条件的修改，以创建新的运行框架。

- Candidate edits are evaluated by regression tests on held-in D_{in} (for testing whether the weakness is resolved) and held-out D_{out} (for checking whether other unknown issues were introduced) splits.
- Candidates are accepted only if they have no regression on both held-in and held-out data.
- Accepted candidates are merged to update the harness to h_{t+1}, while rejected candidates are logged without changing the active harness.

When running `MiniMax M2.5`, `Qwen3.5-35B-A3B`, and `GLM-5` on Terminal-Bench-2, Self-Harness was shown to learn model-specific harness instructions that target at different weaknesses of different base models and improve held-out pass rates.

Self-harness type of work does raise my concerns that if a program is allowed to edit the OS system, abstraction boundaries are broken. The editable surface needs to be properly designed and the permission control and security layers need to live outside this loop. All the challenges around reward hacking still remain.

Agentic Harness Engineering (AHE; Lin et al. 2026) see the bottlenecks of harness evolution are around **observability**—that is, when a rollout fails, we need to know which component is responsible for that and every edit should be grounded by evidence.

The framework creates a closed loop with 3 observability pillars:

1. *Component observability*: every editable harness component has a representation in the file system so the action space is explicit and tracable.
   - A harness contains 7 components: system prompt, tool description, tool implementation, middleware, skill, sub-agent configuration, and long-term memory.
   - Each failure pattern is mapped to one component so the edit can be more targeted.
2. *Experience observability*: analysize and summarize a large amount of raw trajectories into a hierarchy of evidence and failure patterns.
   - Each harness generates k traces.
   - Use an agent (“Agent debugger”) to analysis the trajectories each stored in one file and generate per-task analysis report on the root cause for the failure or success.
   - All the per-task reports are aggregated into a benchmark overview for the next step, and raw traces can be accessed if needed. This layered access structure is more token efficient.
3. *Decision observability*: every edit is paired with a prediction for the next round to validate.
   - An agent (“Evolve agent”) reads the repo and decides which component to edit, and then produces the edit and the reasoning behind it.
   - Every edit is a file-level, falsifiable claim and can be verified in the next round, under two constraints:
     - (1) Edits are only applied to the harness workspace. the runs directory, tracer, verifier, and LLM configuration are read-only, which disables a set of reward hacking (e.g disabling the verifier, swapping the model, or raising the reasoning budget) and thus it can keep every recorded gain attributable to harness edits.
     - (2) Edits are evidence-driven, with a manifesto entry: the failure evidence’s name, the inferred root cause, the targeted fix, and a predicted impact comprising both expected fixes and at-risk regressions.

On Terminal-Bench-2, AHE achieved better than human-designed harness (OpenCode, Terminus-2, Codex) except for Hard tier and a few other self-evolve baselines (ACE, TF-GRPO). The same frozen harness, without further evolving, transfers to SWE-bench-verified, indicating that the evolved harness is able to encode engineering

- 通过在保留内数据划分（用于测试弱点是否已解决）和留出数据划分（用于检查是否引入了其他未知问题）上运行回归测试，评估候选修改。只有在保留内数据和留出数据上均未出现回归时，候选修改
- 才会被接受。被接受的候选修改将被合并，以更新运行框架，而被拒绝的候选修改将被记录，不会更
- 改当前使用的运行框架。

在 Terminal-B 行 `Qwen3 A3B` 和 `GLM-5` 时，结果表明，Self-Harness 能够针对不同基础模型的不同弱点，学习模型专属的运行框架指令，并提高留出集通过率。

Self-Harness 这类工作确实让我担忧：如果允许程序编辑操作系统，抽象边界就会被打破。可编辑范围需要经过合理设计，权限控制和安全层则必须置于这一循环之外。围绕奖励投机的所有挑战依然存在。

智能体运行框架工程（AHE；Lin 等人，2026年）认为，运行框架演化的瓶颈在于**可观测性**——也就是说，当一次轨迹展开失败时，我们需要知道是哪个组件导致了失败，而且每次编辑都应有证据支撑。

该框架通过可观测性的三大支柱构建闭环：

1. 组件可观测性：每个可编辑的运行框架组件都在文件系统中有对应的表示，使动作空间明确且可追踪。
   - 运行框架包含 7 个组件：系统提示词、工具描述、工具实现、中间件、技能 ，
   - 子智能体配置和长期记忆。每种失败模式都映射到一个组件，以便更有针对性地进
   - 行编辑。
2. 经验可观测性：分析并总结大量原始轨迹，将其整理为分层的证据和失败模式。
   - 每个运行框架都会生成追踪记录。使用一个智能体（“智能体调试器”）分析分别存储在单个文件中的轨
   - 迹，并生成逐任务分析报告，说明失败或成功的根本原因。所有逐任务报告都会汇总为基准测试概览，供下一步使用；如有需要，也可以访问原始追踪记录。这种分层访问结构能更高效地利用词元。
   -
3. 决策可观测性：每次编辑都附带一项预测，供下一轮验证。
   - 一个智能体（“演化智能体”）读取代码仓库，决定编辑哪个组件，然后生成编辑内容及其背后的推理依据。每次编辑都是一项文件级的可证伪断言，可在下一轮验证，但须遵守两项约束：(1) 编辑仅应用于运行框架
   - 工作区。运行目录、追踪器、验证器和大语言模型配置均为只读，从而杜绝一类奖励投机行为（例如禁用验证器、更换模型或提高推理预算），确保记录下来的每一项性能提升都可归因于智能体运行框架编辑。(2) 编辑须由证据驱动，并附有宣言条目：失败证据的名称、推断出的根本原因

，

针对性修复，以及涵盖预期修复和潜在回归问题的预测影响。

在 Terminal-Bench-2 上，除困难级别外，AHE 的表现优于人工设计的运行框架（OpenCode、Terminus-2、Codex）以及其他几个自演化基线（ACE、TF-GRPO）。同一个冻结的运行框架无需进一步演化，即可迁移至 SWE-bench-verified，这表明演化后的运行框架能够编码工程

experience into harness components rather than doing benchmark-specific optimization.

Evolutionary Search

Evolutionary search is an optimization method inspired by natural selection (see my old post on evolutionary algorithm). It evolves a population of solutions by mutating them and only keeping those with high "fitness" in the crowd. Evolutionary search comes in handy when (1) the search space is extensive or weirdly shaped; and (2) it is hard to optimize directly with gradients but easy to evaluate solutions. Harness search seems to be a good fit here.

Evolutionary search has been used in prompt engineering in the past studies. **Promptbreeder** (Fernando et al. 2023) optimizes task-specific prompts through a rich set of mutation operations, and interestingly the mutation prompts (i.e. instructions to an LLM to mutate a task prompt) are themselves also improved through evolution. **GEPA** (Agrawal et al. 2025) combines reflection-based prompting with evolutionary search and uses natural language reflection over trajectories of trial and error to propose prompt updates.

Novikov et al. (2025) introduced **AlphaEvolve** as a coding-agent evolutionary search system, which stores a pool of candidate programs and prompts frozen LLMs to generate diffs for improvement. As the system repeatedly evaluates child programs and keeps successful ones, it discovers better solutions in time.

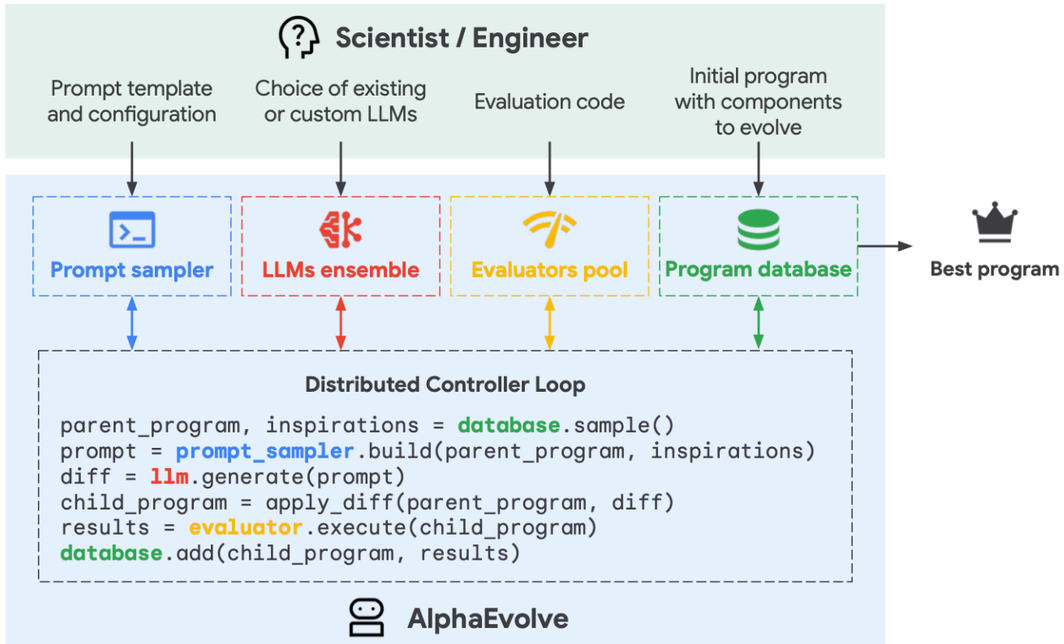


Figure 16: How AlphaEvolve works. (Image source: Novikov et al. 2025)

A few details matter in the design of AlphaEvolve:

- The prompt includes parent programs, results, instructions, and sometimes meta information.
- The coding agent has access to the full repo, but code regions for improvement are explicitly marked with `# EVOLVE-BLOCK-START` and `# EVOLVE-BLOCK-END`.

将经验融入框架组件，而不是进行基准测试专用优化。

演化搜索

演化搜索是一种受自然选择启发的优化方法（参见我之前关于进化算法的文章）。它通过对解进行变异，并仅保留种群中“适应度”高的解，来演化解种群。在以下情况下，演化搜索尤其有用：（1）搜索空间庞大或形状不规则；（2）难以直接通过梯度进行优化，但评估解却很容易。框架搜索似乎很适合采用这种方法。

以往研究已将演化搜索用于提示工程。**Promptbreeder**（Fernando 等人，2023年）通过丰富的变异操作来优化任务专用提示，有趣的是，变异提示（即指示大语言模型对任务提示进行变异的指令）本身也会通过演化得到改进。**GEPA**（Agrawal 等人，2025年）将基于反思的提示方法与演化搜索相结合，并通过对试错轨迹进行自然语言反思来提出提示更新。

Novikov 等人（2025年）提出了**AlphaEvolve**，这是一个编码智能体进化搜索系统，它维护一个候选程序池，并通过提示词引导冻结的大语言模型生成用于改进的差异补丁。随着系统反复评估子代程序并保留成功的程序，它会逐渐发现更好的解决方案。

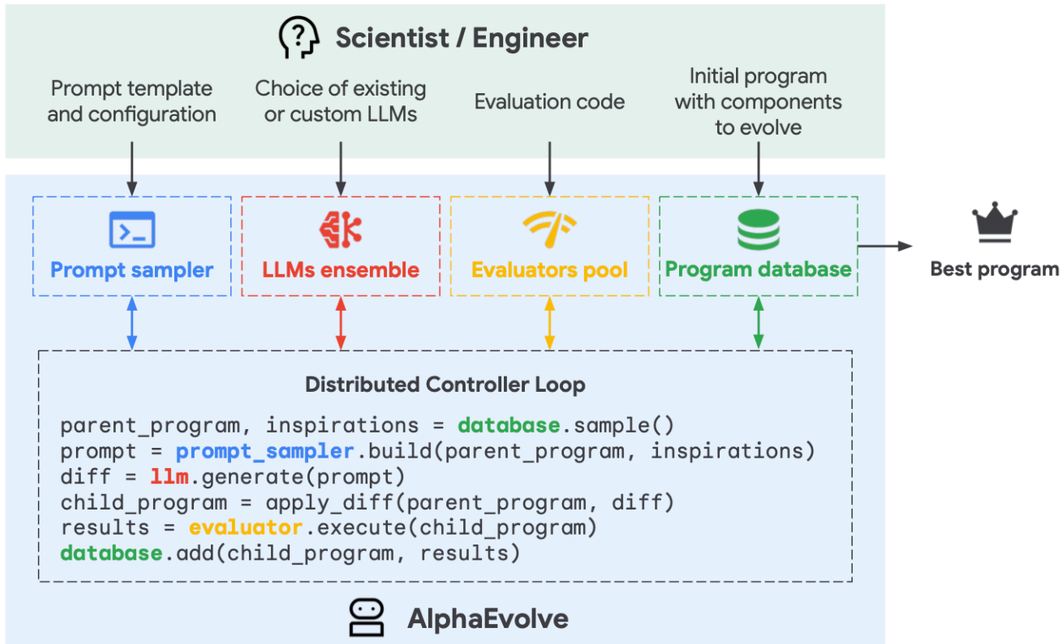


图16：AlphaEvolve 的工作原理。（图片来源：Novikov 等人，2025年）

AlphaEvolve 的设计中有几个重要细节：

- 提示包含亲代程序、结果、指令，有时还包含元信息。编程智能体可以访问整个代码仓库，但需要改进的代
- 码区域会用 `#EVOLVE-BLOCK-START` 和 `# EVOLVE-BLOCK-END` 明确标记。

- Meta-prompt co-evolves with instructions and context as suggested by LLM, in a similar way as how we evolve solution programs.

Ablations show the evolution procedure, context in prompts, meta-prompts, full-file evolution and the use of stronger LLMs.

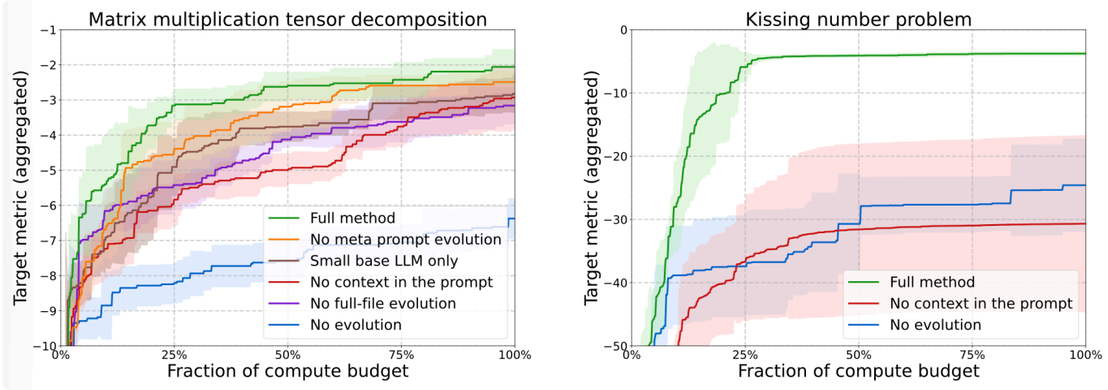


Figure 17: Ablations show the value of everal designs in AlphaEvolve. (Image source: Novikov et al. 2025)

Recent variants such as **ThetaEvolve** (Wang et al. 2025) combines evolutionary search with RL and in-context learning, and **DemoEvolve** (Che, et al. 2026) augments the self-rollout archive with human expert demonstrations as reference experience for harness-level diagnosis and editing. **ShinkaEvolve** (Lange et al. 2025), on the other hand, introduced three new components to improve LLM sampling efficiency:

- More sample-efficient exploration by designing parent sampling to balance performance rank and offspring count.
- Code-novelty rejection sampling by discarding candidates that are too similar to the existing population based on embedding-based cosine similarity.
- Identifying good patterns in successful solutions in a meta-scratchpad to guide future mutation.

Unlike the methods above, which focus on solution improvement, **Darwin Gödel Machine** (DGM; Zhang et al. 2025) explicitly targets the evolution of an editable harness-code repository with an LLM-based coding agent. Precisely, this agent is allowed to modify its own harness. A follow-up work on Hyperagents (Zhang et al. 2026) introduced a meta-agent to control how to modify existing task agents to create new ones.

1. Start with one coding agent in the pool.
2. In each iteration, pick one parent with a probability proportional to its performance and inversely to the number of children it has, to modify and branch off to produce new agents.
3. The selected parent agent examines its own benchmark evaluation log and then proposes improvements to its own harness codebase to generate a new version of the coding agent. Code editing is implemented with two basic tools: (1) bash (args: `<bash_command>`) and (2) editor (args: `view/create/edit <file_path>`).
4. New coding agents are evaluated, and only those with sufficiently high performance are added back into the pool.
5. Repeat steps 2-4 until some stop criteria hit.

- 元提示根据大语言模型的建议，与指令和上下文共同演化，其方式类似于我们演化解答程序的方式。

消融实验展示了演化过程、提示中的上下文、元提示、全文件演化以及使用更强大语言模型的作用。

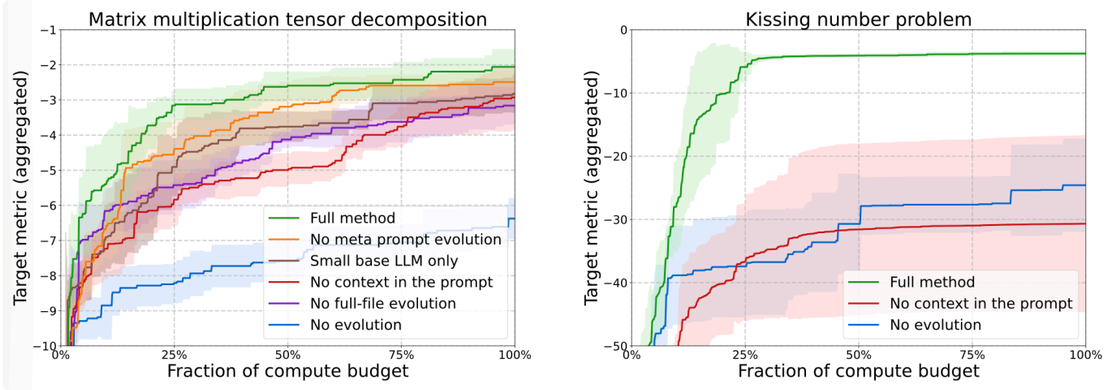


图 17：消融实验展示了 AlphaEvolve 中多项设计的价值。（图片来源：Novikov 等人，2025年）

近期的变体，例如 **ThetaEvolve**（Wang 等人，2025年），将演化搜索与强化学习和上下文学习相结合，而 **DemoEvolve**（Che 等人，2026年）则在人类专家示范的帮助下扩充自生成轨迹档案，将其作为执行框架级诊断和编辑的参考经验。**ShinkaEvolve**（Lange 等人，2025年）则引入了三个新组件，以提高大语言模型采样效率：

- 通过设计兼顾性能排名和后代数量的亲代采样，实现更高样本效率探索。通过基于嵌入的余弦相似度，丢弃与现有种群过于相似的候选项，实现代码新颖性拒绝采样。在元草稿区中识别成功解法中的优秀模式，以指导后续变异。
-

与上述专注于改进解决方案的方法不同，**达尔文哥德尔机**（DGM；Zhang 等人，2025年）明确以利用基于大语言模型的编程智能体来演化可编辑的执行框架代码仓库为目标。具体而言，该智能体获准修改自身的运行框架。后续关于 Hyperagents 的研究（Zhang 等人，2026年）引入了一个元智能体，用于控制如何修改现有任务智能体以创建新的任务智能体。

1. 初始时，池中只有一个编程智能体。2. 每轮迭代中，选择一个父智能体，对其进行修改并分支生成新的智能体；选择概率与其性能成正比，与其子智能体数量成反比。3. 被选中的父智能体检查自身的基准评测日志，然后提出对自身执行框架代码库的改进方案，以生成新版本的编程智能体。代码编辑通过两个基本工具实现：(1) bash（参数：`<bash_command>`）和 (2) 编辑器（参数：`view/create/edit <file_path>`）。4. 对新的编程智能体进行评估，只有性能足够高的智能体才会被加入池中。5. 重复步骤 2-4，直到满足某个停止条件。

DGM is harness evolution under a fixed model. In experiments with `Claude 3.5 Sonnet` as the base LLM and simple initial harness configs, the DGM-discovered agents are comparable to or outperform handcrafted agents on SWE-bench Verified (20% to 50%) and Polyglot (14.2% to 30.7%).

This family of methods works well when candidate solutions are automatically evaluable and candidate fitness is easy to quantify, such as matrix multiplication, GPU kernel optimization, algorithm contests, datacenter scheduling. It struggles with domains where evaluation is slow, ambiguous, or mostly heuristic-based. The compute efficiency and effectiveness of evolution are also concerns.

Joint Optimization with Model Weights

Harness evolution changes the non-parametric system around the model. To enable full self-improvement, the model can totally be allowed to update its own weights at the same time. The weight update can be implemented via improvements in the model training pipeline or continual learning at test time. The topic of continual learning is worthy of its own post in the future.

SIA (Hebbar et al. 2026) is an early attempt to combine harness improvement and model-parameter updates in the same optimization loop, with three components in the design:

- *Meta-Agent*: proposes the initial harness.
- *Task-Specific Agent*: executes the task.
- *Feedback-Agent*: chooses whether to update the harness or the model weights based on recent trajectories.

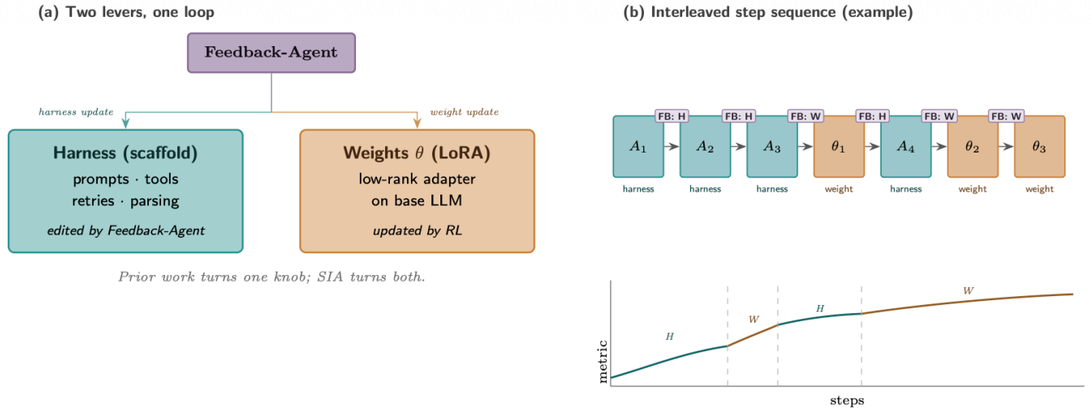


Figure 18: The Feedback-Agent in SIA decides the next iteration type. (Image source: Hebbar et al. 2026)

There are a few confounding choices in SIA's experiments that make the results hard to interpret. For example, the task-specific agent is much weaker than the models used for the Meta-Agent and Feedback-Agent (`gpt-oss-120b` vs `Claude Sonnet 4.6`), and the baselines are too weak to cross-reference cleanly against related methods. I would consider the direction interesting, but the evidence provisional. Yet many challenges, such as training stability and Goodhart effect, still remain open.

Continual Harness (Karten et al. 2026) experimented in long-horizon gameplay setting with harness updating and co-learning a policy model by distilling a strong teacher model's labels on low-reward trajectories.

DGM 是在固定模型下进行的运行框架演化。在以 `Claude 3.5 Sonnet` 型、采用简单初始智能体框架配置的实验中，DGM 发现的智能体在 SWE-bench Verified（20% 至 50%）和 Polyglot（14.2% 至 30.7%）上的表现与手工设计的智能体相当或更优。

当候选解可以自动评估、候选解适应度易于量化时，这类方法表现良好，例如矩阵乘法、GPU内核优化、算法竞赛和数据中心调度。但在评估缓慢、标准模糊或主要依赖启发式方法的领域，这类方法则难以奏效。演化的计算效率和有效性也值得关注。

与模型权重联合优化

运行框架演化改变的是模型周围的非参数系统。要实现全面的自我改进，完全可以允许模型同时更新自身的权重。权重更新可以通过改进模型训练流水线，或在测试时进行持续学习来实现。持续学习这一主题值得在未来单独撰文讨论。

SIA（Hebbar 等，2026年）是在同一优化循环中结合智能体框架改进与模型参数更新的早期尝试，其设计包含三个组件：

- 元智能体：提出初始运行框架。任务专用智能体：执行任务。反馈智能体：根据近期轨迹，选择更新运行框
- 架还是模型权重。
-

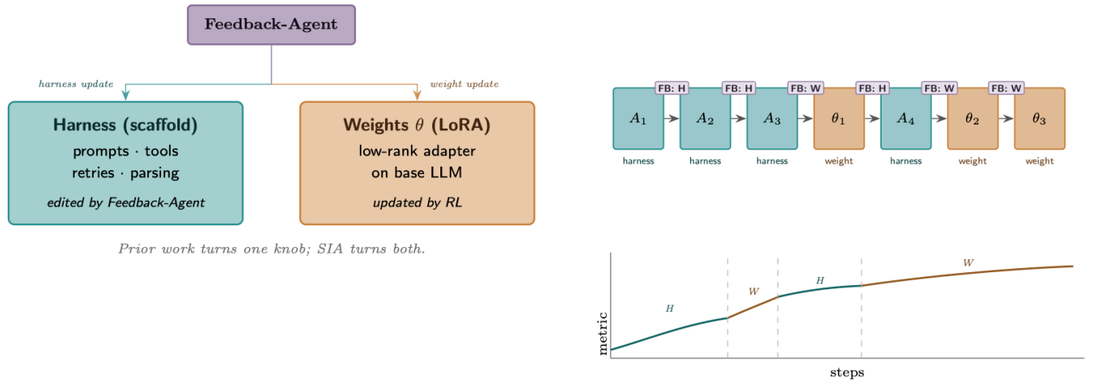


图18：SIA 中的反馈智能体决定下一次迭代的类型。（图片来源：Hebbar 等，2026年）

SIA 的实验中有几项选择引入了混杂因素，使结果难以解读。例如，任务专用智能体所用模型远弱于元智能体和反馈智能体所用的模型（ `gpt-oss-120b` 对比 `Claude Sonnet 4.6`），而且基线过弱，难以与相关方法进行清晰的交叉比较。我认为这一方向很有意思，但现有证据尚不足以下定论。此外，训练稳定性和古德哈特效应等诸多挑战仍未解决。

Continual Harness（Karten 等，2026年）在长时程游戏交互场景中开展实验，通过蒸馏强大教师模型对低奖励轨迹给出的标签，进行智能体框架更新并协同学习策略模型。

Future Challenges

The AI Scientist line of work is a strong demonstration that an expert-designed harness can coordinate a large portion of auto-research loop, experimented in the form of writing research papers. But paper production is not identical to scientific discovery. A system can write a plausible manuscript while still having fabricated citations, implementation drift, or weak experimental results.

Trehan & Chopra (2026) tested whether LLMs can go from a research idea to a paper with minimal scaffolding and basic tools (i.e., `read_file` , `write_file` , `llm_search` , `list_files`). Each idea had a dedicated workspace where agents could generate and read documents as part of context. They experimented in three domains (world models, multi-agent RL, AI safety & alignment), with each domain containing 45-50 high-quality seed documents to inspire new ideas. Only four ideas were selected by human experts to run through the full pipeline, and only one was fully executed into a paper. They observed six recurring failure modes in the experiments:

- *Bias toward training-data defaults*: use old libraries, stale commands, standard formats, or assumptions not grounded in the actual repository or dataset.
- *Implementation drift under execution pressure*: when implementation becomes technically complex, the model may move toward a common simpler solution rather than the proposed method.
- *Memory and context degradation*: long-horizon projects lose critical details unless logs are written as persistent artifacts.
- *Over-optimism*: the model declares success despite noisy or failed experiments, similarly observed as “p-hacking and eureka-ing” pattern by Bubeck et al. (2025) where models can introduce “numerical duct tape” and declare victory when signals are still noise.
- *Insufficient domain intelligence*: the model lacks tacit craft knowledge, e.g. predicting implementation complexity, judging whether an experimental result is plausible, or knowing which baselines matter.
- *Weak scientific taste*: experiments may be executable but fail to answer the right question.

Toward full RSI, researchers have made real progress, but several bottlenecks remain.

1. Weak and fuzzy evaluators. Many research claims do not have a fast and precise verifier, and the same is true for many real-world tasks. Current self-improvement loops work best for tasks when evaluation metrics are measurable and objective, similar as how RL works.

Research taste, novelty, and long-term scientific value are much harder to measure. For example, research taste often mixes problem framing, experimental design, and judgment about which surprising results are worth pursuing and which failure cases are worth retries.

2. Context and memory lifecycle. Memory grows as AI agents become more autonomous and independent. A useful harness needs to manage context and memory to complement existing limitation in long-context generation while still maximizing the success of long-horizon tasks. Since humans are able to maintain memory through our life time, I see an anoloy here that context engineering will and should become a core part of intelligence, rather than staying in the software system layer.

3. Negative results. Researchers are incentivized to publish successful results and thus literature is biased toward successes. LLMs trained on a vast amount of data (mostly human created, at least for now, lol) may be bad at deciding when to abandon a hypothesis, report a negative result, or even acknowledge a failure due to the imablance of success vs failure cases in data. A research harness should make failed attempts easy to preserve, as learning from failure is the best way to trim down the task search space.

未来挑战

AI Scientist 系列研究以撰写研究论文的形式开展实验，有力地证明了专家设计的运行框架能够协调自动化研究循环中的大部分环节。但产出论文并不等同于科学发现。一个系统即使能写出看似可信的稿件，仍可能存在伪造引用、实现漂移或实验结果缺乏说服力等问题。

Trehan 与 Chopra（2026年）测试了大语言模型能否仅凭极少的脚手架和基础工具（即 `read_file` 、 `write_file` 、 `llm_search`、 `list_files` ），将研究想法转化为论文。每个想法都有专属的工作空间，智能体可以在其中生成和阅读文档，将其作为上下文的一部分。他们在三个领域（世界模型、多智能体强化学习、人工智能安全与对齐）开展了实验，每个领域都包含 45–50 篇高质量的种子文档，用于启发新想法。人类专家仅选出了四个想法，让其走完整条流水线，而最终只有一个想法被完整落实为论文。他们在实验中观察到了六种反复出现的失效模式：

- 偏向训练数据默认模式：使用旧版程序库、过时命令、标准格式，或采用未以实际代码仓库或数据集为依据的假设。执行压力下的实现漂移：当实现在技术上变得复杂时，模型可能转向常见的、更简单的解决方案，而非
- 采用所提出的方法。记忆与上下文退化：长期项目会丢失关键细节，除非将日志写成持久化制品。过度乐观：即使实验噪声很大或已经失败，模型仍宣告成功。Bubeck 等人（2025年）也观察到了类似的“p值操纵与顿悟式宣告”模式：模型可能引入“数值补丁”，并在信号仍只是噪声时就宣告胜利。领域智能不足：模型缺乏隐
- 性实践知识，例如预测实现复杂度、判断实验结果是否合理，或知道哪些基线重要。科学品味欠佳：实验可能可以执行，却未能回答正确的问题。
-
-
-

在迈向完全递归自我改进的过程中，研究人员已取得切实进展，但仍存在若干瓶颈。

1. 能力薄弱且标准模糊的评估器。许多研究论断缺乏快速、精确的验证器，许多现实世界任务也是如此。当前，自我改进循环在评估指标可衡量且客观的任务上效果最佳，这与强化学习的工作原理类似。

研究品味、新颖性和长期科学价值则难以衡量得多。例如，研究品味往往融合了问题界定、实验设计，以及对哪些出乎意料的结果值得深入探索、哪些失败案例值得重试的判断。

2. 上下文与记忆生命周期。随着人工智能智能体变得更加自主、独立，记忆也会不断增长。一个实用的运行框架需要管理上下文与记忆，以弥补长上下文生成现有的局限，同时尽可能提高长时程任务的成功率。人类能够终生保持记忆，我认为这里存在一个类比：上下文工程将会且应当成为智能的核心组成部分，而不是停留在软件系统层。

3. 阴性结果。研究人员受到激励，倾向于发表成功的结果，因此文献也偏向于呈现成功。大语言模型使用海量数据进行训练（至少目前大部分还是人类创作的，哈哈），由于数据中的成功案例与失败案例比例失衡，它们可能不善于判断何时该放弃一个假设、报告阴性结果，甚至承认失败。研究运行框架应当让失败的尝试易于保存，因为从失败中学习是缩小任务搜索空间的最佳方式。

4. Diversity collapse. Evolutionary and RL loops tend to exploit known high-reward patterns. We need mechanisms to prevent the population from collapsing into variants of the same solution. This is especially critical for open-ended research, where the best path may initially look worse under the current evaluator.

5. Reward hacking. A self-improvement loop optimizes whatever signal it is given. If the reward comes from unit tests, the agent may overfit to tests; if it comes from a judge model, it may learn reward hacking tricks specific to this judge; if it comes from benchmark scores, it may exploit benchmark artifacts.

The evaluator and permission control should likely sit outside the loop that evolves harness, with held-out tests, trace audits, and human review at decision points that matter—how much oversight can be scaled up and automated remains an open research area.

6. Long-term success. An extrinsic loop of optimization works on rewards outside of individual rollouts that we can simulate in training sandbox.

Take coding agent as an example. Coding agents have already increased daily productivity in software engineering, but many optimization goals are still too short-term. It can often complete the task at hand, but less obvious how it should protect the long-term health of a repo collectively maintained by hundreds or thousands of engineers. Standard sandbox-based RLVR-style training rarely captures maintainability, ownership boundaries, migration cost, backwards compatibility, or future debugging burden.

7. The role of humans. Humans should move up the stack, not be removed from the loop, meaning that human should provide oversight at the right time, at the right abstraction level and our system design should consider when and how to set up such touch points.

Many challenges listed above need human's feedback and steering. After all, we are building the technology for better future of humanity, not other way around.

Citation

Please cite this work as:

> Weng, Lilian. "Harness Engineering for Self-Improvement". Lil'Log (Jul 2026).
> https://lilianweng.github.io/posts/2026-07-04-harness/

Or use the BibTeX citation:

```
@article{weng2026harness,
  title = {Harness Engineering for Self-Improvement},
  author = {Weng, Lilian},
  journal = {lilianweng.github.io},
  year = {2026},
  month = {July},
  url = "https://lilianweng.github.io/posts/2026-07-04-harness/"
}
```

Appendix: Some useful benchmarks

- **PaperBench**: replicate 20 ICML 2024 Spotlight and Oral papers from scratch, including understanding paper contributions, developing a codebase, and successfully executing experiments.

4. 多样性坍塌。演化和强化学习循环往往倾向于利用已知的高奖励模式。我们需要机制来防止种群坍塌为同一解决方案的不同变体。这对于开放式研究尤为关键，因为在当前评估器的评判下，最佳路径最初可能看起来反而更差。

5. 奖励投机。自我改进循环会针对给定的任何信号进行优化。如果奖励来自单元测试，智能体可能会对测试过拟合；如果奖励来自评判模型，它可能会学会专门针对该评判模型的奖励投机技巧；如果奖励来自基准测试分数，它可能会利用基准测试伪影。

评估器和权限控制或许应当置于演进运行框架的循环之外，并在关键决策节点引入留出测试、轨迹审计和人工审核——监督能够在多大程度上规模化和自动化，仍是一个有待探索的研究领域。

6. 长期成功。外部优化循环针对的是单次轨迹展开之外的奖励，而这些轨迹展开可以在训练沙箱中模拟。

以编程智能体为例。编程智能体已经提高了软件工程的日常生产力，但许多优化目标仍然过于短期化。它通常能够完成眼前的任务，但对于如何维护一个由数百乃至数千名工程师共同维护的代码仓库的长期健康状态，答案却不那么明确。标准的基于沙箱的可验证奖励强化学习式训练很少能涵盖可维护性、权属边界、迁移成本、向后兼容性或未来的调试负担。

7. 人类的角色。人类应当向更高层级迈进，而不是被排除在环路之外。这意味着，人类应在恰当的时机、恰当的抽象层级上提供监督，而我们的系统设计应考虑何时以及如何设置这类交互节点。

上述许多挑战都需要人类反馈和引导。毕竟，我们开发技术是为了让人类拥有更美好的未来，而不是反过来。

引文

请按以下格式引用本文：

> 翁荔。"用于自我改进的运行框架工程"。Lil'Log（2026年7月）。
> https://lilianweng.github.io/posts/2026-07-04-harness/

或使用 BibTeX 引文：

```
@article{weng2026harness,
title = {Harness Engineering for Self-Improvement},
author = {Weng, Lilian},journal = {lilianweng.github.io},
year = {2026},month = {July},
url = "https://lilianweng.github.io/posts/2026-07-04-harness/"}
```

附录：一些实用的基准测试

- **PaperBench**：从零开始复现 20 篇2024年国际机器学习大会聚焦报告与口头报告论文，包括理解论文贡献、开发代码库以及成功执行实验。

- Each replication task is decomposed into smaller, individually gradable tasks.
- 8,316 rubrics in total, co-developed with the paper authors.
- The best model at the time (`Claude 3.5 Sonnet` , ~21%) does not outperform ML PhDs.
- Includes PaperBench, PaperBench Code-Dev (a lighter version), and JudgeEval.

- **CORE-Bench**: evaluate computational reproducibility of published research.
  - 270 tasks based on 90 scientific papers across computer science, social science, and medicine.
  - Tasks involve reproducing results from provided code and data.
  - Includes multiple difficulty levels and both language-only and vision-language tasks.
  - The best reported agent at the time (`GPT-4o` and `GPT-4o-mini`) achieved only 21% accuracy on the hardest task.

- **ScienceAgentBench**: evaluate LLM agents for data-driven scientific discovery.
  - Extracts 102 tasks from 44 peer-reviewed publications in four disciplines (math, chemistry, biology, geography).
  - Covers basic data-science tasks in these domains: data processing, model development, data analysis, and information visualization.

- **RE-Bench**: evaluate frontier AI agents on realistic ML research-engineering envs against human experts.
  - 7 challenging, open-ended ML research-engineering environments.
  - Each environment = (scoring function, starting solution, reference solution); each can be run with 8 or fewer H100 GPUs.
  - Examples: optimize a kernel, run a scaling-law experiment, fix an embedding, fine-tune GPT-2 for QA, etc.
  - Includes data from 71 eight-hour attempts by 61 distinct human experts.
  - Human experts achieved non-zero score in 82% of 8-hour attempts; 24% matched or exceeded strong reference solutions.
  - Best AI agents scored 4× higher than humans at a 2-hour budget, but humans had better returns to longer budgets and exceeded agents at 8-hour and 32-hour settings.

- **MLE-bench**: evaluate ML engineering agents on offline Kaggle competitions.
  - Contains 75 ML-engineering competitions curated from Kaggle.
  - Tests training models, preparing datasets, running experiments, and submitting predictions to grading scripts.
  - Uses Kaggle public leaderboards as human baselines.
  - Best setup in the paper, `o1-preview` with AIDE scaffolding, reached at least Kaggle bronze-medal level in 16.9% of competitions.
  - Includes resource-scaling and contamination analyses.

- **KernelBench**: evaluate correctness and speed for generated GPU kernels.
  - 250 PyTorch tasks to evaluate whether LLM can write fast and correct kernels.
  - The evaluation metric fast_p = the percentage of generated kernels that are correct and faster than baseline.

- 每项复现任务都被分解为可单独评分的较小任务。共有 8,316 项评分标准，与论文作者共同制定。当时表现最好的模型（ `Claude 3.5 Sonnet` ，~21%）并未超越机器学习博士。包括 PaperBench、PaperBench Code-Dev（轻量版）和 JudgeEval。

- **CORE-Bench**：评估已发表研究的计算可复现性。
  - 基于计算机科学、社会科学和医学领域的 90 篇科学论文构建的 270 项任务。任务涉及使用提供的代码和数据复现结果。涵盖多个难度级别，包括纯语言任务和视觉语言任务。当时报告的最佳智能体（ `GPT-4o` 和 `GPT-4o-mini` ）在最难任务上的准确率仅为 21%。

- **ScienceAgentBench**：评估大语言模型智能体开展数据驱动的科学发现的能力。
  - 从四个学科（数学、化学、生物学、地理学）的 44 篇同行评审出版物中提取了 102 项任务。涵盖这些领域的基础数据科学任务：数据处理、模型开发、数据分析和信息可视化。

- **RE-Bench**：评估前沿人工智能智能体在真实机器学习研究工程环境中的表现，并与人类专家进行比较 .
  - 7 个具有挑战性的开放式机器学习研究工程环境。每个环境 = （评分函数、初始解决方案、参考解决方案）；每个环境均可使用不超过 8 块 H100 图形处理器运行。示例：优化内核、开展缩放定律实验、修复嵌入、微调 GPT-2 以执行问答任务等。包含 61 位不同人类专家进行的 71 次八小时尝试的数据。在八小时尝试中，人类专家有 82% 的尝试获得了非零分数；24% 的尝试达到或超过了高水平参考解决方案。表现最佳的人工智能智能体在两小时时间预算下的得分比人类高 4× ，但随着时间预算增加，人类获得的收益更大，并在八小时和三十二小时的设置下超越了智能体。

- **MLE-bench**：通过离线 Kaggle 竞赛评估机器学习工程智能体。
  - 包含从 Kaggle 精选的 75 场机器学习工程竞赛。测试训练模型、准备数据集、运行实验以及向评分脚本提交预测结果的能力。使用 Kaggle 公开排行榜作为人类基线。论文中表现最佳的配置为 `o1-preview` 搭配 AIDE 脚手架，在 16.9% 的竞赛中至少达到了 Kaggle 铜牌水平。包含资源扩展和数据污染分析。

- **KernelBench**：评估生成的GPU内核的正确性和速度。
  - 250 项 PyTorch 任务，用于评估大语言模型能否编写快速且正确的内核。评估指标 fast_p = 生成的内核中正确且速度快于基线的内核所占的百分比。

References

[1] Good, I. J. “Speculations Concerning the First Ultraintelligent Machine.” *Advances in Computers*, 6:31–88, 1965.

[2] Yudkowsky, Eliezer. “Recursive Self-Improvement.” LessWrong, 2008.

[3] Choi, et al. “Anchored Self-Play for Code Repair.” ICML 2026.

[4] Zhao, et al. “Absolute Zero: Reinforced Self-play Reasoning with Zero Data.” arXiv preprint arXiv:2505.03335, 2025.

[5] Yuan, et al. “Self-Rewarding Language Models.” arXiv preprint arXiv:2401.10020, 2024.

[6] Chen, et al. “Self-Play Fine-Tuning Converts Weak Language Models to Strong Language Models.” ICML 2024.

[7] Zhang, et al. “Agentic Context Engineering: Evolving Contexts for Self-Improving Language Models.” ICLR 2026.

[8] Ye, et al. “Meta Context Engineering via Agentic Skill Evolution.” arXiv preprint arXiv:2601.21557, 2026.

[9] Lee, et al. “Meta-Harness: End-to-End Optimization of Model Harnesses.” arXiv preprint arXiv:2603.28052, 2026.

[10] Lu, et al. “Towards end-to-end automation of AI research.” *Nature*, 651:914–919, 2026.

[11] Meng, et al. “ScientistOne: Towards Human-Level Autonomous Research via Chain-of-Evidence.” arXiv preprint arXiv:2605.26340, 2026.

[12] Kulikov, et al. “Autodata: An agentic data scientist to create high quality synthetic data.” arXiv preprint arXiv:2606.25996, 2026.

[13] Hu, Lu, and Clune. “Automated Design of Agentic Systems.” ICLR 2025.

[14] Madaan, et al. “Self-Refine: Iterative Refinement with Self-Feedback.” NeurIPS 2023.

[15] Zhang, et al. “AFlow: Automating Agentic Workflow Generation.” ICLR 2025.

[16] Zelikman, et al. “Self-Taught Optimizer (STOP): Recursively Self-Improving Code Generation.” COLM 2024.

[17] Zhang, et al. “Self-Harness: Harnesses That Improve Themselves.” arXiv preprint arXiv:2606.09498, 2026.

[18] Fernando, et al. “Promptbreeder: Self-Referential Self-Improvement Via Prompt Evolution.” arXiv preprint arXiv:2309.16797, 2023.

[19] Agrawal, A. et al. “GEPA: Reflective Prompt Evolution Can Outperform Reinforcement Learning.” arXiv preprint arXiv:2507.19457, 2025.

[20] Novikov, et al. “AlphaEvolve: A coding agent for scientific and algorithmic discovery.” arXiv preprint arXiv:2506.13131, 2025.

[21] Lange, Imajuku, and Cetin. “ShinkaEvolve: Towards Open-Ended And Sample-Efficient Program Evolution.” arXiv preprint arXiv:2509.19349, 2025.

[22] Wang, et al. “ThetaEvolve: Test-time Learning on Open Problems.” arXiv preprint arXiv:2511.23473, 2025.

参考文献

[1] 古德，I. J.《关于第一台超智能机器的推测》。 计算机进展，6:31–88，196 5.

[2] 埃利泽·尤德科夫斯基。《递归自我改进》。 LessWrong，2008年。

[3] Choi 等。《用于代码修复的锚定自博弈》。 2026年国际机器学习大会。

[4] Zhao 等。《Absolute Zero：零数据强化自博弈推理》。 arXiv 预印本 arXiv:2505.03335，2025年。

[5] Yuan 等。《自奖励语言模型》。 arXiv 预印本 arXiv:2401.10020，2024年。[6] Chen 等。《自博弈微调将弱语言模型转化为强语言模型》。 2024年国际机器学习大会。[7] Zhang 等。《智能体上下文工程：为自我改进语言模型演化上下文》。 2026年国际学习表征会议

.

[8] Ye 等。《通过智能体技能演化实现元上下文工程》。 arXiv 预印本 arXiv:2601.21557，2026年。[9] Lee 等。《Meta-Harness：模型运行框架的端到端优化》。 arXiv 预印本 arXiv:2603.28052，202 6.

[10] Lu 等人。“迈向人工智能研究的端到端自动化。” 自然，651:914–919，2026年。

[11] Meng 等人。“ScientistOne：通过证据链迈向人类水平自主研究。” arXiv 预印本 arXiv:2605.26340，2026年。

[12] Kulikov 等人。“Autodata：用于创建高质量合成数据的智能体数据科学家。” arXiv 预印本 arXiv:2606.25996，2026年。

[13] Hu、Lu 和 Clune。“智能体系统自动化设计。” 国际学习表征会议，2025年。[14] Madaan 等人。“Self-Refine：通过自我反馈进行迭代改进。” 神经信息处理系统大会，2023年。[15] Zhang 等人。“AFlow：实现智能体工作流生成的自动化。” 国际学习表征会议，2025年。[16] Zelikman 等人。“自学优化器（STOP）：递归自我改进的代码生成。” 语言建模会议，2024年。[17] Zhang 等人。“Self-Harness：能够自我改进的智能体运行框架。” arXiv 预印本 arXiv:2606.09498，2026年。[18] Fernando 等人。“Promptbreeder：通过提示词进化实现自指式自我改进。” arXiv 预印本 arXiv:2309.16797，2023年。

[19] Agrawal, A. 等。“GEPA：反思式提示词进化可超越强化学习。” arXiv 预印本 arXiv:2507.19457，2025年。

[20] Novikov 等。“AlphaEvolve：用于科学与算法发现的编程智能体。” arXiv 预印本 arXiv:2506.13131，2025年。

[21] Lange、Imajuku 和 Cetin。“ShinkaEvolve：迈向开放式且样本高效的程序进化。” arXiv 预印本 arXiv:2509.19349，2025年。

[22] Wang 等人。“ThetaEvolve：针对未解问题的测试时学习。” arXiv 预印本 arXiv:2511.23473，2025年。

[23] Zhang, et al. “Darwin Gödel Machine: Open-Ended Evolution of Self-Improving Agents.” arXiv preprint arXiv:2505.22954, 2025.

[24] Zhang, et al. “Hyperagents.” arXiv preprint arXiv:2603.19461, 2026.

[25] Yuksekgonul, et al. “Learning to Discover at Test Time.” arXiv preprint arXiv:2601.16175, 2026.

[26] Riaz, et al. “Epistemic Uncertainty for Test-Time Discovery.” arXiv preprint arXiv:2605.11328, 2026.

[27] Hebbar, et al. “SIA: Self Improving AI with Harness & Weight Updates.” arXiv preprint arXiv:2605.27276, 2026.

[28] Trehan and Chopra. “Why LLMs Aren’t Scientists Yet: Lessons from Four Autonomous Research Attempts.” arXiv preprint arXiv:2601.03315, 2026.

[29] Bubeck, et al. “Early science acceleration experiments with GPT-5.” arXiv preprint arXiv:2511.16072, 2025.

[30] Starace, et al. “PaperBench: Evaluating AI’s Ability to Replicate AI Research.” ICML 2025.

[31] Wijk, et al. “RE-Bench: Evaluating frontier AI R&D capabilities of language model agents against human experts.” ICML 2025.

[32] Chan, et al. “MLE-bench: Evaluating Machine Learning Agents on Machine Learning Engineering.” arXiv preprint arXiv:2410.07095, 2024.

[33] Chen, et al. “ScienceAgentBench: Toward Rigorous Assessment of Language Agents for Data-Driven Scientific Discovery.” ICLR 2025.

[34] Siegel, et al. “CORE-Bench: Fostering the Credibility of Published Research Through a Computational Reproducibility Agent Benchmark.” TMLR 2024.

[35] Ouyang, et al. “KernelBench: Can LLMs Write Efficient GPU Kernels?” arXiv preprint arXiv:2502.10517, 2025.

[36] Lin, et al. “Harness Updating Is Not Harness Benefit: Disentangling Evolution Capabilities in Self-Evolving LLM Agents.” arXiv preprint arXiv:2605.30621, 2026.

[37] Lin, et al. “Agentic Harness Engineering: Observability-Driven Automatic Evolution of Coding-Agent Harnesses.” arXiv preprint arXiv:2604.25850, 2026.

[38] Karten, et al. “Continual Harness: Online Adaptation for Self-Improving Foundation Agents.” arXiv preprint arXiv:2605.09998, 2026.

[39] Che, et al. “DemoEvolve: Overcoming Sparse Feedback in Agentic Harness Evolution with Demonstrations.” arXiv preprint arXiv:2605.24539, 2026.

[23] Zhang 等人。“达尔文哥德尔机：自我改进智能体的开放式演化。” arXiv 预印本 arXiv:2505.22954，2025年。

[24] Zhang 等人。《Hyperagents》。 arXiv 预印本 arXiv:2603.19461，2026年。

[25] Yuksekgonul 等人。《学习在测试时进行发现》。 arXiv 预印本 arXiv:2601.16175，2026年。

[26] Riaz 等人。《用于测试时发现的认知不确定性》。 arXiv 预印本 arXiv:2605.11328，2026年。

[27] Hebbar 等人。《SIA：通过运行框架与权重更新实现自我改进人工智能》。 arXiv 预印本 arXiv:2605.27276，2026年。

[28] Trehan 和 Chopra。《为什么大语言模型还不是科学家：四次自主研究尝试的经验教训》。arXiv 预印本 arXiv:2601.03315，2026年。

[29] Bubeck 等人。《利用 GPT-5 开展的早期科学加速实验》。 arXiv 预印本 arXiv:2511.16072，2025年。

[30] Starace 等人。《PaperBench：评估人工智能复现人工智能研究的能力》。国际机器学习大会，2025年。

[31] Wijk 等人。《RE-Bench：对照人类专家评估语言模型智能体的前沿人工智能研发能力》。国际机器学习大会，2025年。

[32] Chan 等人。《MLE-bench：评估机器学习智能体的机器学习工程能力》。arXiv 预印本 arXiv:2410.07095，2024年。

[33] Chen 等人。《ScienceAgentBench：迈向对面向数据驱动的科学发现的语言智能体的严格评估》。国际学习表征会议，2025年。

[34] Siegel 等人。《CORE-Bench：通过计算可复现性智能体基准提升已发表研究的可信度》。机器学习研究汇刊，2024年。

[35] Ouyang 等人。《KernelBench：大语言模型能编写高效的GPU内核吗？》arXiv 预印本 arXiv:2502.10517，2025年。

[36] Lin 等人。“智能体框架更新不等于智能体运行框架收益：厘清自演化大语言模型智能体的演化能力。” arXiv 预印本 arXiv:2605.30621，2026年。

[37] Lin 等人。“智能体运行框架工程：编程智能体运行框架的可观测性驱动的自动演化。”arXiv 预印本 arXiv:2604.25850，2026年。

[38] Karten 等人。“Continual Harness：面向自改进基础智能体的在线适应。” arXiv 预印本 arXiv:2605.09998，2026年。

[39] Che 等人。“DemoEvolve：利用示范克服智能体运行框架演化中的稀疏反馈问题。”arXiv 预印本 arXiv:2605.24539，2026年。